University of Stuttgart
Germany

Faculty of Electrical Engineering and Computer Science
Computational Cognitive Science

Toward Fortran Level AMICA in Julia: Block Based Learning and GPU Acceleration

Valentin Morlock

Master of Science thesis in Software Engineering — March 2026

Supervisors

Main Supervisor:	Benedikt Ehinger	Computational Cognitive Science Faculty of Electrical Engineering and Computer Science, Department of Computer Science

Abstract

Adaptive Mixture of Independent Component Analyzers (AMICA) is a powerful maximum-likelihood based method for Independent Component Analysis (ICA) and is widely used in EEG artifact removal. AMICA.jl is a promising re-implementation of the Fortran reference in the Julia programming language that improves the accessibility of the algorithm's implementation but cannot yet match the convergence behavior, performance, and numerical stability of the reference.

Our work examines whether AMICA.jl can be improved to match the Fortran reference in correctness and performance while still offering a more accessible and extensible code base. By exporting intermediate values from a modified Fortran implementation and comparing them against Julia, multiple implementation differences were identified and corrected, resulting in closely matching outputs. Numerical stability was improved, allowing AMICA.jl to reliably work with 32-bit precision. To improve performance and potentially outperform AMICA Fortran, we added GPU acceleration, introduced blockwise processing, and implemented multithreading in AMICA.jl.

We performed an extensive benchmark series measuring memory use and runtime, and showed that our improved Julia implementation substantially improved both metrics. When compared to the Fortran reference, AMICA.jl is now competitive in terms of memory use and CPU runtime, and achieves faster runtimes when using GPU acceleration.

Our work therefore positions AMICA.jl as a correct, practically viable and more accessible alternative to the Fortran reference implementation.

Acknowledgements

I would like to sincerely thank my supervisor, Benedikt Ehinger, for the interesting discussions, his valuable feedback, and support throughout my work, as well as for introducing me to Julia and AMICA in the first place. Working on this project was a fascinating experience, and it allowed me to explore a range of topics that I would probably never have encountered otherwise.

I am deeply grateful to my parents and friends for their constant encouragement, for always taking the time to talk, and for motivating me to keep going during the course of this thesis.

My deepest thanks go to Katharina for her support during the more challenging times, for her patience and understanding during the many long evenings when this work demanded much of my time and attention, and for making sure that this thesis became not only better in content, but also better in appearance.

Contents

Abstract **iii**

Acknowledgements **vii**

Contents **ix**

List of Abbreviations **xi**

1 Introduction **1**
1.1 EEG Artifact Removal 2
1.2 Independent Component Analysis 3
1.3 AMICA 4
1.3.1 AMICA.jl 6
1.3.2 GPU Programming in Julia 7
1.4 Summary 9

2 Results **11**
2.1 Approach 11
2.1.1 Benchmarking 11
2.1.2 Testing 14
2.2 Implementation 15
2.2.1 Aligning results of AMICA.jl to AMICA Fortran 15
2.2.2 GPU Support 16
2.2.2.1 Tradeoffs of Broadcasts versus Custom Kernels . 17
2.2.2.2 GPU port of AMICA.jl 21
2.2.3 Blockwise Processing 22
2.2.4 Object Pool 24
2.2.5 Multithreading 26

2.3 Benchmarks 28
2.3.1 Correctness 28
2.3.2 Runtime 30
2.3.3 Memory Usage 35
2.3.4 Bottlenecks 42

3 Discussion **43**
3.1 Correctness and Convergence 43
3.2 Runtime 44
3.3 Memory 44
3.4 Practical implications 45
3.5 Limitations 46
3.5.1 Gaps in the Implementation 46
3.5.2 Gaps in the Benchmark 47

4 Summary **49**

Bibliography **51**

A Appendices **55**
A.1 Parameterizations to be Benchmarked 55
A.2 Full Results of the Runtime Benchmarks 57
A.3 Full Results of the Memory Benchmarks 59

List of Abbreviations

AMICA Adaptive Mixture of Independent Component Analyzers

BSS Blind Source Separation

EEG Electroencephalography

EM Expectation Maximization

GSMs Gaussian Scale Mixtures

ICA Independent Component Analysis

LL Logarithmic Likelihood

LLM Large Language Model

PDF Probability Density Function

RSS Resident Set Size

1

Introduction

When engaging in a conversation within a room full of other people speaking, the human brain can effortlessly separate the voice of the conversation partner from auxiliary sounds in real time. However, achieving the same separation using digital signal processing is surprisingly complicated. An often cited exemplary scenario, named the "cocktail party problem", is the aforementioned room of multiple speakers, recorded by several microphones. Each microphone records a mixture of the individual sound sources present in the observed room, at varying sound levels. (Naik & Kumar, 2011)

Separating the unknown source signals within the room given the signal mixtures is called Blind Source Separation (BSS). An exemplary problem is shown in Figure 1.1, where the left three graphs represent the mixed source signals x, as recorded by the microphones, which can be unmixed to the unmixed source signals displayed on the right, as originally emitted by the sound sources. The graphs make it visible how the three sources influence the mixtures by varying degrees. BSS finds application in various fields of signal processing, for example in medical imaging, sound processing or Electroencephalography (EEG) artifact removal. (Choi et al., 2005; Naik & Kumar, 2011)

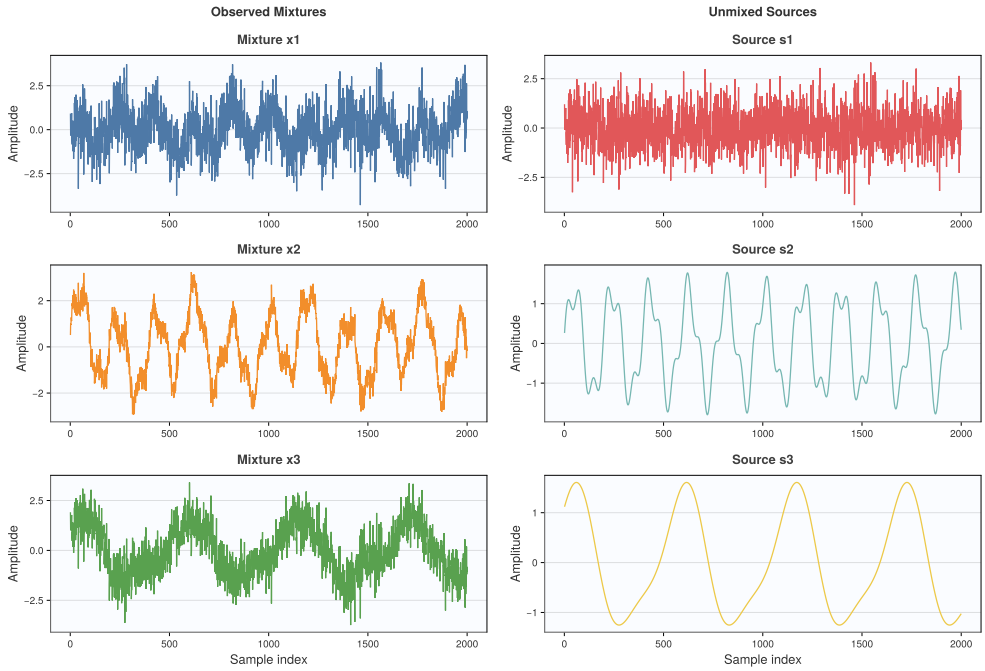

Figure 1.1 – Schematic visualization of observed mixtures x and unmixed sources s

1.1 EEG Artifact Removal

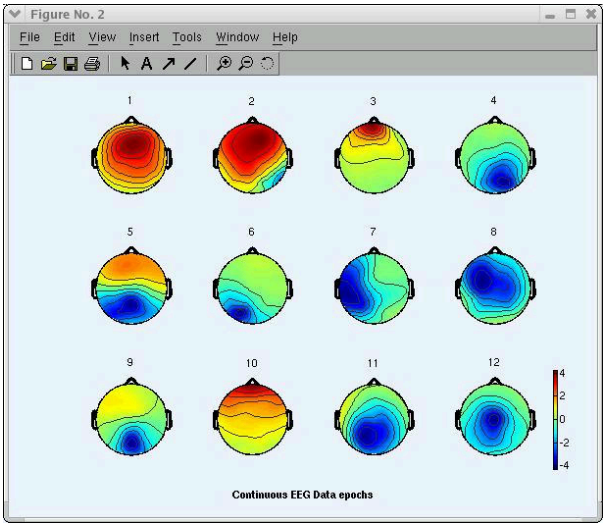

Figure 1.2 – Active scalp areas by separated source, created by the EEGLAB software (*Artifact Removal in EEGLAB Using ICA*, n.d.)

EEG recordings are extensively used in scientific research as well as clinical diagnostics. The goal of an EEG usually is to create a recording of the signals emitted by certain areas of the subject's brain. The subject has electrodes attached to their scalp, and each electrode is capturing a separate recording of the electrical signals observed on the scalp, often called a channel. Different sources might emit these electric signals, and therefore each electrode's recording consists of a distinct mixture of these source signals.

A recording typically does not contain only the signals emitted by the brain but also other unwanted signals, e.g. caused by the blinks or movements of an eye. The distortions caused by this are called artifacts. EEG artifact removal is a class of approaches to remove such artifacts from the EEG recordings. Artifact removal can be seen as a BSS problem where the recorded channels contain mixtures of neural activity and undesired sources. After separating the recording's sources using BSS, sources with relevant information are kept and sources with artifacts are discarded. (Urigüen & Garcia-Zapirain, 2015)

1.2 Independent Component Analysis

Several classes of algorithms to solve BSS are successfully employed in scientific and industrial applications, one of the most popular classes being Independent Component Analysis (ICA).

BSS is called blind as very little or no prior knowledge about the sources or the mixing process is known. The problem is therefore highly under-determined and separation is only possible by imposing additional assumptions like statistical independence, sparseness or non-gaussianity that constrain the solution space. (Yu et al., 2013)

ICA assumes the mixtures to be a linear combination of the source signals and unmixes the signals such that they are as statistically independent from each other as possible. Formally speaking, the observations x are modelled as linear mixtures $x = A * s$ and an optimal unmixing matrix $W = A^{-1}$ is being determined. (Choi et al., 2005; Lee, 1998; Yu et al., 2013)

The process of ICA can be visualized as *rotating* the mixtures versus the sources. Different specific approaches on how to determine the optimal unmixing exist. Some algorithms maximize the entropy of the signal while others, like Adaptive

Mixture of Independent Component Analyzers (AMICA), maximize the likelihood of the unmixed data. (Pearlmutter & Parra, 1996; Stone, 2004)

1.3 AMICA

The algorithm AMICA as introduced by Palmer et al. (2012) is a popular variant of ICA, especially in the area of EEG signal processing. It has been shown to be one of the best-performing ICA algorithms in the field of artifact removal on EEG sources (Delorme et al., 2012). AMICA is an ICA algorithm based on maximum likelihood estimation, which extends other maximum likelihood based ICA approaches (e.g., Infomax and extended Infomax) in the following two ways.

Following Palmer et al. (2012), instead of assuming a fixed Probability Density Function (PDF) for the source signals, AMICA estimates the density of the source signals $q_i(s_i(t))$, where i indexes the source component, as a mixture of m Gaussian Scale Mixtures (GSMs) q_{ij} parametrized by location μ_{ij}, scale β_{ij} and shape ρ_{ij}, where j indexes the mixture component for source i.

The mixing of the GSMs is defined by Equation (1.1) which additionally includes a shift of the components by μ_{ij} and scaling by $\sqrt{\beta_{ij}}$ where $0 < \beta_{ij} < 2$, which is required to ensure non-gaussianity. A visualization can be seen in Figure 1.3.

$$q_i(s_i(t)) = \sum_{j=1}^{m} \alpha_{ij} \sqrt{\beta_{ij}} q_{ij}\Big(\sqrt{\beta_{ij}}\big(s_i(t) - \mu_{ij}\big); \rho_{ij}\Big) \tag{1.1}$$

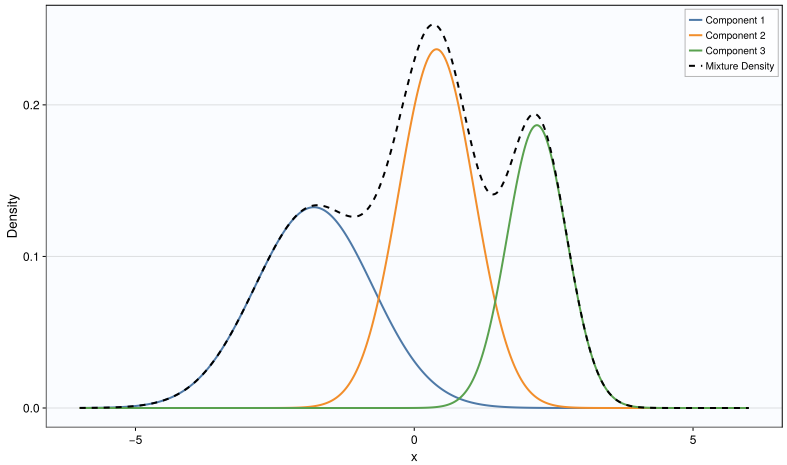

Figure 1.3 – Visualization of how multiple gaussians are combined to a PDF

Based on the estimated source $y_t = Wx_t$ and the density model (defined above) $q_i(y_i)$, the algorithm computes the negative Logarithmic Likelihood (LL) $L(W)$ of the observed data under the current W and the current source density model. The LL is then minimized. (Palmer et al., 2012)

$$\begin{aligned} f(y_t) &= \sum_i (-\log q_i(y_{it})) \\ L(W) &= \sum_{t=1}^{N} -\log|\det W| + f(y_t) \end{aligned} \tag{1.2}$$

Therefore, the single model AMICA estimates $\{W, \alpha_{ij}, \mu_{ij}, \beta_{ij}, \rho_{ij}\}$.

Second, AMICA includes support for estimating multiple models which improves unmixing in scenarios where the source signals are non-stationary. A model index $h \in \{1..M\}$ is added to the parameters of the source density model and to the unmixing matrix W. γ_h indicates the probability that a model is active at time t, and exactly one model is active for each t. Each model is centered by c_h. (Palmer et al., 2012)

This extends these estimated parameters to $\{W_h, c_h, \gamma_h, \alpha_{hij}, \mu_{hij}, \beta_{hij}, \rho_{hij}\}$ where

$$x(t) = A_h s(t) + c_h \tag{1.3}$$

To optimize those parameters, the AMICA algorithm uses Expectation Maximization (EM) to iteratively assign samples to models and source-mixture components and then updates the unmixing matrix and density-parameters.

There are currently multiple implementations of AMICA. Palmer (n.d.) initially implemented it in Matlab (*AMICA - Matlab Implementation*, n.d.), and later added the current de facto standard implementation in Fortran. Lulkin (2023) created the initial version of AMICA.jl, an AMICA implementation written in Julia. AMICA Fortran performs well and has advanced features like multi-threading or distributed processing. However, it is difficult to maintain, compile and extend and lacks compatibility with programming languages commonly used in science like Python or Julia. The implementation in Matlab is more approachable, but has worse performance in terms of convergence quality and computational efficiency. The implementation in Julia, modeled after the Matlab version, suffers from those same issues.

1.3.1 AMICA.jl

AMICA.jl, initially implemented by Lulkin (2023), is an implementation of the AMICA algorithm modelled after the Matlab implementation (*AMICA - Matlab Implementation*, n.d.). Both implement the basic AMICA algorithm as well as multi-model functionality.

Compared to the multi threaded Fortran implementation, AMICA.jl, as present at the start of this project, solves the AMICA algorithm around 25x slower on a single thread and omits block based learning as well as parallelization across cores or nodes. Convergence and numerical stability are inferior compared to the reference implementation. As visualized in Figure 1.4, before this project, AMICA.jl computed significantly different results and aborted after 5 iterations.

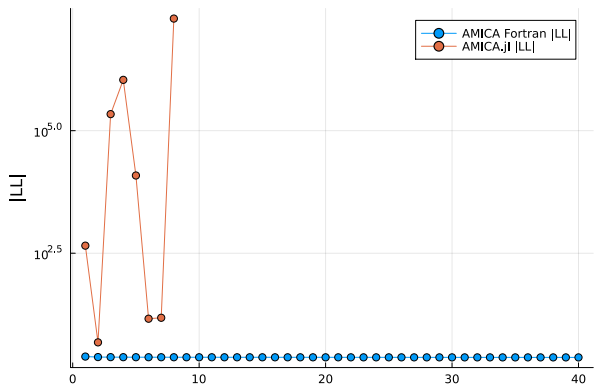

Figure 1.4 – LL of AMICA.jl and AMICA Fortran on the "Memorize" reference dataset. We plot the absolute value on a logarithmic scale to show the large deviation before AMICA.jl aborts with `NaN`.

Despite the aforementioned issues, AMICA.jl remains promising because it is implemented in a more approachable programming language and in a significantly more compact and better structured codebase. This would allow a wider audience to work with, understand, and extend the AMICA algorithm.

The logical next step would therefore be to align the results to the Fortran code, and to achieve performance that is on par with or surpasses the reference. One obvious step to achieve similar performance is adding parallel processing on multiple cores of a CPU. The Fortran code implements this by dividing the dataset into blocks which are then processed in parallel.

GPUs offer significantly more effective parallelization than CPUs. Running AMICA on a GPU would therefore present a viable way to surpass the performance of all currently existing implementations. Since applying AMICA to large datasets can currently take several hours, this could substantially improve its practical usefulness.

1.3.2 GPU Programming in Julia

While Julia code runs on the CPU by default, several Julia packages provide powerful abstractions to port existing code to the GPU. Broadly speaking, there are two levels of abstraction:

1. **High level** abstractions provided by libraries like CUDA.jl (*Cuda.jl*, n.d.) or AMDGPU.jl (*Amdgpu.jl*, n.d.) expose custom array types such as `CuArray` which behave like regular Julia arrays but store data in the GPU memory and run operations performed on those data types on the GPU. This allows using common Julia operations like vector or matrix multiplications, broadcasts, map, reduce etc. on the GPU. In addition to that, vendor provided libraries for AMD and Nvidia GPUs expose precompiled GPU kernels for common operations like matrix multiplication, which work with the aforementioned array types. While simple to implement, these GPU integrations are limited in terms of which computations are supported. (*Julia for HPC - GPU Programming*, n.d.)

```jl
using CUDA

A = CuArray([0,1,2,3,4,5,6])
A .+= 1 # runs on GPU
```

Listing 1.1 – High level gpu abstraction in julia, adapted from (*Julia for HPC - GPU Programming*, n.d.)

An important benefit of this approach is that the implementation is independent of the GPU backend: the same computation could work with a CPU array or a GPU array of another backend.

2. **Lower level** abstractions provide more control on how computations are executed on the GPU by writing custom kernels in Julia. The CUDA.jl package provides a `@cuda` macro which runs arbitrary code on the GPU, however, work needs to be split across GPU threads manually, by calling the `threadIdx` function to obtain the index of the current GPU thread and then perform work on the right block of data. Similar abstractions exist for other GPU backends, e.g. Metal.jl for Apple Silicon GPUs or AMDGPU.jl for AMD GPUs. This method of writing custom kernels implies writing a separate kernel per GPU backend, which increases implementation effort and limits reusing code. To simplify this, the KernelAbstractions.jl (*Kernelabstractions.jl*, n.d.) package provides utilities to efficiently parallelize work on the GPU without manually managing blockwise computations. (*Julia for HPC - GPU Programming*, n.d.)

```jl
using KernelAbstractions

@kernel function vadd!(C, @Const(A), @Const(B))
    i = @index(Global)
    @inbounds C[i] = A[i] + B[i]
end

function my_vadd!(C, A, B)
    backend = get_backend(A)
    kernel! = vadd!(backend)
    kernel!(C,A,B, ndrange = size(C))
end
```

Listing 1.2 – Lower level gpu abstraction in julia (*Julia for HPC - GPU Programming*, n.d.)

To summarize, Julia offers two backend agnostic ways to implement computations on the GPU: using broadcasts or a custom KernelAbstractions kernel. Raimondo et al. (2012) have demonstrated that it is possible to run Infomax-ICA on a GPU, and the AMICA Fortran implementation shows that the AMICA algorithm can also be parallelized across threads and nodes. Since AMICA, similar to Infomax-ICA, calculates expensive operations such as matrix multiplications or exponents on each sample of the dataset, we expect AMICA to make good use of the massive parallelization of a GPU. Combined with the GPU programming capabilities in Julia, this suggests that adding GPU support to AMICA.jl is a feasible direction for further work.

1.4 Summary

ICA is a class of algorithms to solve BSS problems, with AMICA being the currently best performing algorithm in the area of EEG artifact removal. AMICA.jl is an implementation of the algorithm in the Julia programming language, which tries to make AMICA more approachable, but currently suffers from inferior accuracy, numerical stability and performance. All implementations existing before this work require extensive compute and, on large EEG datasets, exhibit runtimes up to several hours, which makes running AMICA intensive in time and cost.

This work therefore explores how an optimized Julia implementation of AMICA can match the Fortran reference, measured by convergence quality and runtime efficiency, and which algorithmic changes, like block-based learning and GPU acceleration, most strongly improve performance.

2 Results

Throughout the implementation phase of this work, we iteratively updated, tested and benchmarked the AMICA.jl implementation. The following chapter describes the approach we took, the results we obtained, the conclusions we draw, and the final version of AMICA.jl at which we arrived.

2.1 Approach

2.1.1 Benchmarking

To improve performance and correctness, an appropriate benchmarking methodology had to be established first. Throughout the implementation phase of this work, we performed benchmarks on the 71 channel, 319,500 sample reference EEG dataset "Memorize" as published on the Palmer (n.d.) page. For the final benchmarks we also use a roughly 7x smaller 19 channel, 172,704 sample EEG dataset from Barras & Booth (2026), subject 1, which we call "Cognitive Workload" as well as an around 7x larger 128 channel, 1,260,379 sample "EEG Eye Tracking" dataset from Schepers et al. (2026).

All benchmarks were run on the same machine with specifications as shown in Table 2.1 and, unless specified otherwise, the algorithm was executed for 40 iterations with the Newton Method enabled in all iterations.

CPU	2x AMD EPYC 7452 32-Core Processor, 354GB Memory
GPU	1x NVIDIA RTX A6000, 49GB Memory

Table 2.1 – Specification of the server used for benchmarking AMICA.jl

To evaluate performance of the implementations, we measured (1) time spent per iteration, i.e. total runtime divided by the number of iterations, (2) peak CPU memory and (3) peak GPU memory used.

We defined the CPU memory as the Resident Set Size (RSS) reported as `VmRSS` in `/proc/<pid>/status` and GPU memory as the value reported by the function `nvml_gpu_mem_for_pid` of the NVIDIA NVML library. For both values, we captured the whole process tree to avoid missing subprocesses e.g. launched by the MPI library used in Fortran. We continuously sampled both memory values every 100ms using a Julia script and took the maximum observed value. In theory, this approach might miss very short memory spikes, but due to the high number of samples, we considered it robust enough to compare different AMICA variants.

The first iteration of the AMICA algorithm takes longer than all subsequent iterations, particularly in the Julia implementation. Furthermore, both implementations perform various preprocessing steps before the first iteration begins. In our benchmarks over 40 iterations, this initial time significantly distorts the average iteration time. Real-world scenarios often involve 1,000 or more iterations, so this factor is negligible for practical use. Therefore, we measure the iteration time in all runtime benchmarks starting from iteration 1 as $\frac{\text{time_after_iter_1}}{\text{n_iter} - 1}$.

	Memory	Time
CPU	7.5 GB	6.0 s/iter

Where relevant, we show the performance of the current implementation step in a sidebar table. The table alongside this paragraph shows the initial performance numbers measured for AMICA.jl upon the start of the project.

To find performance bottlenecks and make targeted improvements, we additionally implemented fine grained benchmarks using the `TimerOutputs.jl` library (Carlsson, n.d.). As shown in Figure 2.1 and Listing 2.1, the macro

`@timeit` allows measuring the time spent and memory allocated in a certain code area and hierarchically visualizes total and average measurements.

```jl
@timeit to "initialize_shape_parameter!" initialize_shape_parameter!(myAmica, lrate)

@timeit to "removeMean" if remove_mean
    removed_mean = removeMean!(data)
end

@timeit to "sphering" if do_sphering
    S = sphering!(data)
    myAmica.S = S
    myAmica.LLdetS = logabsdet(S ▷ Array)[1]
else
    myAmica.S = I
    myAmica.LLdetS = 0
end
```

Listing 2.1 – Sample usage of the TimerOutputs.jl package

		Time			Allocations		
Tot / % measured:		121s / 6.8%			8.65GiB / 10.2%		
Section	ncalls	time	%tot	avg	alloc	%tot	avg
sphering	3	467ms	5.6%	156ms	175MiB	19.4%	58.2MiB
update_parameters	3	2.82s	34.1%	941ms	174MiB	19.3%	58.0MiB
kernel	3	2.82s	34.1%	941ms	174MiB	19.3%	58.0MiB
dA	3	421ms	5.1%	140ms	174MiB	19.3%	58.0MiB
dmu_denom	3	220ms	2.7%	73.3ms	5.86KiB	0.0%	1.95KiB
dbeta_denom	3	227ms	2.7%	75.8ms	3.16KiB	0.0%	1.05KiB
kp	3	287ms	3.5%	95.6ms	3.16KiB	0.0%	1.05KiB
dlambda_numer	3	268ms	3.2%	89.3ms	3.05KiB	0.0%	1.02KiB
dmu_numer	3	239ms	2.9%	79.8ms	3.02KiB	0.0%	1.01KiB
drho_numer	3	1.13s	13.6%	376ms	2.88KiB	0.0%	981B
sum_z	3	32.6ms	0.4%	10.9ms	2.88KiB	0.0%	981B
prop	3	33.7μs	0.0%	11.2μs	5.72KiB	0.0%	1.91KiB
para	3	32.0μs	0.0%	10.7μs	4.06KiB	0.0%	1.35KiB
shape	3	36.8μs	0.0%	12.3μs	32.0B	0.0%	10.7B
loc	3	8.32μs	0.0%	2.77μs	32.0B	0.0%	10.7B
scale	3	5.70μs	0.0%	1.90μs	32.0B	0.0%	10.7B
update_sources	3	67.9ms	0.8%	22.6ms	174MiB	19.3%	58.0MiB
removeMean	3	523ms	6.3%	174ms	174MiB	19.3%	58.0MiB
movetogpu	3	160ms	1.9%	53.3ms	173MiB	19.2%	57.7MiB
calculate_u_and_Lt	3	2.50s	30.2%	833ms	17.4MiB	1.9%	5.79MiB
kernel	3	2.40s	29.0%	799ms	17.3MiB	1.9%	5.76MiB
ldet	3	1.00ms	0.0%	335μs	94.2KiB	0.0%	31.4KiB
qconst	3	22.9μs	0.0%	7.62μs	2.88KiB	0.0%	981B
Q	3	98.8ms	1.2%	32.9ms	176B	0.0%	58.7B
sum	3	137μs	0.0%	45.6μs	176B	0.0%	58.7B

Figure 2.1 – Sample output of the TimerOutputs.jl package, ordered by average allocation

2.1.2 Testing

The first step of this work was to achieve algorithmic correctness with the AMICA Fortran implementation as baseline. The AMICA algorithm computes the LL of the current unmixing for each iteration. As it is simpler to compare the single LL value instead of the whole unmixing matrix, we chose to use the LL as a proxy to validate whether two implementations of AMICA compute matching results. While it is theoretically possible that two different unmixings result in the exact same LL, we consider that highly unlikely and therefore a limitation we accepted.

However, comparing just the algorithm's output makes it hard to identify and fix errors in the implementation. We therefore added a test suite which compares not only the output but also intermediary values, allowing us to identify the root cause of mismatching results. To do so, we created a modified Fortran implementation which runs for one iteration and writes a set of intermediary values to binary files. This version is built and executed within a Julia test suite, which then runs AMICA.jl and compares the binary files to the Julia implementation.

We have also set up a GitHub Actions pipeline that runs the test suite every time a commit is pushed to the repository, so that every contributor can run the tests without having to install Fortran dependencies on their local computer. This also makes it immediately clear if a pull request fails the tests.

This proved helpful during the initial bug fixing of AMICA.jl and also during the implementation of performance improvements, as regressions could be identified early.

2.2 Implementation

2.2.1 Aligning results of AMICA.jl to AMICA Fortran

At the beginning of this project, AMICA.jl returned results which didn't match the output of AMICA Fortran, and took more iterations to converge. In addition to that, `NaN` values caused frequent early termination and hinted that there might be issues with numerical stability. The goal of this phase was therefore to identify which parts of the algorithm are implemented differently in AMICA.jl.

Comparing the two implementations can be tedious, especially given that variables are named differently and the Fortran code can be rather cryptic. To support that process, we made use of Large Language Models (LLMs), specifically "Claude 4.5 Sonnet" using the "Claude Code" tool, to identify where in the Fortran code a specific value is computed and to extract a mathematical representation of the computation implemented in Fortran. As we checked all changes using the aforementioned test suite, the LLM output could be readily verified for correctness.

We iteratively changed AMICA.jl to pass the test suite, which resulted in a LL that closely tracks AMICA Fortran, as shown in Figure 2.2.

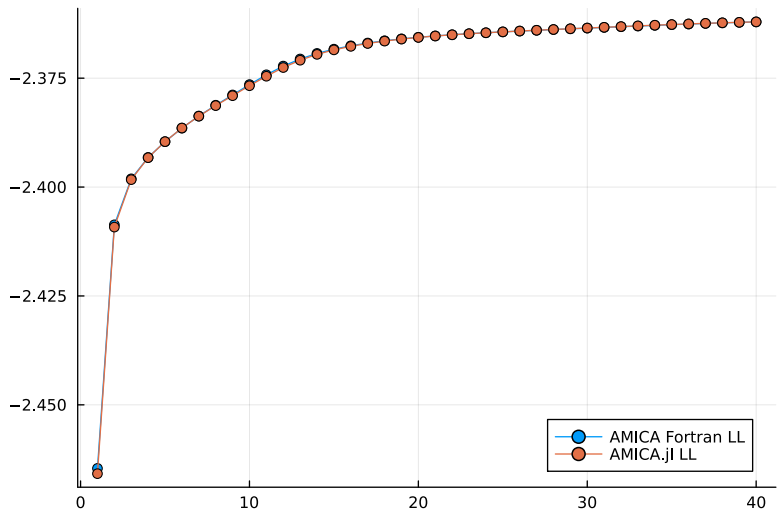

Figure 2.2 – AMICA.jl LL closely tracking AMICA Fortran over 40 iterations

Around 15 algorithmic changes were required to align results with version 17 of the Fortran implementation. Some were small changes to the implementation of the algorithm, e.g. removing a square root, subtracting 1.0 from an exponent or flipping a sign in the newton update logic, while other areas like the update of parameter β required a complete rewrite. Some of those issues reflected bugs and inconsistencies within the AMICA.jl implementation, whereas others reflect inconsistencies between the Matlab and Fortran reference implementations.

	Memory	Time
CPU	7.5 GB	6.0 s/iter
CPU	3.5 GB	7.1 s/iter

The fixes we made slightly increased iteration times but already cut the memory usage in half compared to the initial AMICA.jl, as shown in the sidebar.

In addition to that, we made improvements to numerical stability not present in AMICA Fortran, for example clamping certain values to a small ε to avoid division by zero. In our tests, this enables AMICA.jl to reliably operate on `Float32` data - a significant performance improvement not present in the Fortran implementation.

2.2.2 GPU Support

Broadly speaking, there are two approaches to perform computations on the GPU in Julia - either by using custom kernels or by using broadcast operations, both having distinct advantages and disadvantages. Both, however, require using the array type specific to the GPU backend.

To allow computing AMICA on the GPU while still working on the CPU, we had to make the `SingleModelAmica` struct generic over the kind of array being used. An excerpt of the adjusted code can be seen in Listing 2.2. A notable limitation of Julia is that a type parameter can't be further parametrized, so a separate parameter was required for each distinct number of dimensions used within `SingleModelAmica`.

```jl
# before
mutable struct SingleModelAmica{T} <:AbstractAmica
  location::Array{T, 2}
  shape::Array{T, 2}
  Lt::Array{T, 1}
  # ... further fields ...
end

# after
mutable struct SingleModelAmica{
  T,
  Array1<:DenseArray{T,1},
  Array2<:DenseArray{T,2},
  Array3<:DenseArray{T,3}
} <: AbstractAmica
  location::Array2
  shape::Array2
  Lt::Array1
  # ... further fields ...
end
```

Listing 2.2 – Adaptation of `SingleModelAmica` to support different array types

2.2.2.1 Tradeoffs of Broadcasts versus Custom Kernels

Given the adjusted type, many broadcast operations within AMICA.jl work with GPU vectors without further changes. However, all places using scalar indexing had to be replaced by either broadcasts or custom kernels. Similarly, certain library functions are not supported by some or all GPU backends. For example, the `Base.inv` function works with CUDAs `CuArray` type but fails when passed Metal's `MtlArray` type. In cases where no alternatives could be found, we

temporarily move data to the CPU memory and perform the computation on the CPU.

```jl
ldet = -logabsdet(A ▷ Array)[1]
```

Listing 2.3 – The `logabsdet` function does not support GPU arrays

While implementing GPU support, we noticed that the main tradeoff between broadcasts and custom kernels lies in the reuse of intermediary values and in undesired allocations. A broadcast is simple to understand and implement, but it usually operates on the whole dataset. Therefore, using a result in more than one subsequent calculation requires either computing it twice or allocating the intermediary value.

We illustrate this tradeoff using the example in Listing 2.4, which implements a computation using scalar indexing and a `for` loop. It computes the value `fp` once and reuses it to compute `dlambda_numer` and `g`. As `fp` is written to a scalar stack variable, no heap memory is allocated. Summation happens by incrementing both result variables.

```jl
g .= zero(T)
dlambda_numer .= zero(T)
for k, i, j in size(y)
  fp = y_rho[k, i, j] * sign(y[k, i, j]) * shape[i, j]
  dlambda_numer[i, j] += z[k, i, j] * (fp * y[k, i, j] - T(1.0))^2
  g[k, i, j] += scale[i, j] * z[k, i, j] * fp
end
```

Listing 2.4 – Computing `g` and `dlambda_numer` in a `for` loop

One way to run this computation on the GPU is to use broadcasts as in Listing 2.5. `fp` is calculated for all data points and stored in a three dimensional array. The accumulation in the loop is replaced by the `sum` function.

```jl
  fp = y_rho .* sign.(y) .* push_dimension(shape)
  dlambda_numer = sum(z .* (fp .* y .- T(1.0)) .^ 2, dims=1)[1, :, :]
  g .= sum(push_dimension(scale) .* z .* fp, dims=3)[:, :, 1]
```

Listing 2.5 – Computing `g` and `dlambda_numer` using broadcasts

In addition to `fp`, Julia implicitly allocates the values passed to the `sum` function. This approach therefore causes three allocations of size $\text{samples} * \text{mixtures} * \text{m}$, which consumes significant time and memory. This can be improved by reusing a `temporary` / `scratch` array as shown in Listing 2.6, but peak memory consumption is still $2 * \text{samples} * \text{mixtures} * \text{m}$ higher than in the initial version from Listing 2.4.

```jl
fp = y_rho .* sign.(y) .* push_dimension(shape)
scratch = similar(y)
scratch .= z .* (fp .* y .- T(1.0)) .^ 2
dlambda_numer = sum(scratch, dims=1)[1, :, :]
scratch .= push_dimension(scale) .* z .* fp
g .= sum(scratch, dims=3)[:, :, 1]
```

Listing 2.6 – Computing `g` and `dlambda_numer` using broadcasts and a scratch array

This shows that implementing such computations with loop-like semantics is beneficial in terms of memory consumption. The tool to operate on individual datapoints, comparable to a loop, on a GPU is a custom kernel, which leads to the question whether such operations would be better implemented as a custom kernel. The critical part here is the two summing operations. As the kernel is started within hundreds or thousands of parallel GPU threads, it is no longer possible to simply add to the output variables as in Listing 2.4. Instead, some kind of synchronization logic is required to avoid data corruption when multiple threads operate on the same variable, which is commonly implemented using the Atomix.jl package (*Atomix.jl*, n.d.).

```jl
@kernel function g_dlambda_kernel!(
    g, dlambda_numer,
    @Const(y_rho), @Const(y), @Const(shape), @Const(scale), @Const(z), @Const(T)
)
    k, i, j = @index(Global, NTuple)
    fp = y_rho[k, i, j] * sign(y[k, i, j]) * shape[i, j]
    Atomix.@atomic dlambda_numer[i, j] += z[k, i, j] * (fp * y[k, i, j] - T(1.0))^2
    Atomix.@atomic g[k, i, 1] += scale[i, j] * z[k, i, j] * fp
end
g .= zero(T)
dlambda_numer .= zero(T)
kernel! = g_dlambda_kernel!(get_backend(y))
kernel!(g, dlambda_numer, y_rho, y, shape, scale, z, T, ndrange=size(y))
```

Listing 2.7 – Computing `g` and `dlambda_numer` using a custom KernelAbstractions.jl kernel

In terms of memory efficiency, the kernel implementation (Listing 2.7) shares the benefits of the loop implementation. However, the two atomic operations negate most of the performance benefits of GPU parallelization, as the high number of locking operations is rather inefficient. In addition to that, the order of summing is no longer deterministic, which causes inconsistent results due to floating point inaccuracies and decreases numerical stability. Practically speaking, that can result in `NaN` values within the accumulated values.

One might ask how the built-in `sum` operation handles summing in an efficient and deterministic way on the GPU. CUDA.jl implements accumulation using a hierarchical summing algorithm. The algorithm roughly works as follows:

1. Within each GPU processing block, one thread loads a batch of data to a block local variable
2. The threads of the block hierarchically accumulate the local data as shown in Listing 2.8, until the local sum is found in `scratch[1]`. Due to the `thread ⩽ d` condition, each loop iteration halves the number of threads that perform the summing

3. `scratch[1]` is written to another vector, with one element per GPU block, and a second kernel is launched which sums over the outputs of the former step.

```jl
while d > 0
  @synchronize()
  if thread ≤ d
      ai = offset * (2 * thread - 1)
      bi = offset * (2 * thread)
      scratch[bi] += scratch[ai]
  end
  offset = offset * 2
  d = d ÷ 2
end
```

Listing 2.8 – Schematic representation of the hierarchic summing step

We considered implementing this algorithm in our custom kernels, but ultimately found that we were not able to match the performance of the built-in CUDA.jl implementation. As this approach significantly increases the complexity of our implementation, we decided it was not worth further investigation.

A further limitation of the kernel implementation is that CPU performance in our tests wasn't on par with a comparable broadcast or loop implementation, probably due to the overhead of dispatching a high number of tasks.

2.2.2.2 GPU port of AMICA.jl

Based on the discussed tradeoff between broadcasts and kernels, we chose the following design when adding GPU support to AMICA.jl.

The **initial preparation** logic, consisting of subtracting the means, sphering and parameter initialization, runs on the CPU. As those steps are executed only once and typically neither memory use nor duration is a bottleneck, porting them to the GPU was not a priority.

The main logic of the AMICA algorithm is computed on the GPU using broadcasts. There are two exceptions for which we decided to use custom kernels, which we discuss in more detail below.

Updating `z` and `LL` in a kernel has the advantage that `logsumexp_Q` can be used to update `z` and `LL` without allocating `logsumexp_Q`, `Qmax` or `z_sum`, all of them having the number of samples as one of their dimensions. As the only accumulation is for the per sample likelihood `LL`, only $m * \text{mixtures}$ locking operations happen for each sample and lock congestion thus is not as much of an issue compared to other accumulations.

Computing the newton method parameter `B` in a kernel has two advantages. First, `B` is defined differently for diagonal vs non diagonal elements, which is difficult to implement using broadcasts. Second, the `posdef` value is based on an intermediary computation used to calculate `B`, which would be hard to access when implemented as broadcast. As no summing happens and `posdef` is only ever flipped from `true` to `false`, no locking is required.

	Memory	Time
CPU	7.5 GB	6.0 s/iter
CPU	3.5 GB	7.1 s/iter
GPU	15.6 GB	0.1 s/iter
CPU	3.6 GB	6.6 s/iter

Benchmarks of this implementation are promising. Time per iteration is reduced by around a factor of 70, but memory usage is still lagging behind the Fortran implementation, especially when running on GPU. The test suite still passes and the resulting LL matches Fortran.

2.2.3 Blockwise Processing

After adding GPU acceleration, we found that the high memory use of AMICA.jl might be a prohibitive factor in running it on larger datasets, as especially GPU memory is often constrained. To improve on this, we implemented a pattern also found in AMICA Fortran, called blockwise processing.

Accumulation is critical to the AMICA algorithm. Several steps follow a similar pattern: First, a value is computed for each sample, component and mixture. The results of this computation are then accumulated per component and mixture.

As discussed before and shown in Listing 2.4, the most memory efficient way of implementing such computations is a loop, which however is hard to implement in a performant way on GPU or to parallelize in general.

Blockwise processing is a compromise between full dataset broadcasts and a single loop. The dataset is divided into a number of blocks, which are then processed independently. Each block is transformed and accumulated. The result of the accumulation is then added to the final result. If blocks are processed

in sequence, this theoretically reduces the required amount of memory relative to block size versus full problem size.

```jl
# without blocks
scratch .= (z .* (y_rho .* sign.(y) .* shape))
dmu_numer .= sum(scratch, dims=1)[1, :, :]

# with blocks
dmu_numer .= zero(T)

for blocks in 1:num_blocks
  range = ((block-1)*block_size+1):(block*block_size)
  scratch .= (z[range] .* (y_rho[range] .* sign.(y[range]) .* shape[range]))
  dmu_numer .+= sum(scratch, dims=1)[1, :, :]
end
```

Listing 2.9 – Schematic representation of blockwise processing

To implement blockwise processing in AMICA.jl, we first had to change the code structure to allow certain steps to run per block before using the results to update the gaussian parameters and the mixing matrix. To do that, we pulled the computation of `z`, `LL`, `y`, `y_rho` and `source_signals` into the method previously called `update_parameters`. This allows us to directly use the values computed for the current block, effectively making them local variables within the block loop body. We also reduced the dimensions of the corresponding fields in the `SingleModelAmica` to the configured block size.

Using this approach and a block size of 1000 samples, memory consumption of the AMICA algorithm is reduced to around 1GB of memory. Compared to the previous benchmark, GPU performance suffers, which is expected given that the GPU can no longer parallelize over the whole dataset but is instead limited to a number of threads equal to the block size. The former 0.1 s iteration time for GPU executions is however still achievable using a block size of 10.000 (0.13 s / iteration), so introducing blockwise processing has not caused a regression of the optimal GPU performance. Setting an appropriate block size is, in the GPU case, a tradeoff between memory use and processing time.

	Memory	Time
CPU	7.5 GB	6.0 s/iter
CPU	3.5 GB	7.1 s/iter
GPU	15.6 GB	0.1 s/iter
CPU	3.6 GB	6.6 s/iter
GPU	0.9 GB	0.5 s/iter
CPU	1.1 GB	4.9 s/iter

2.2.4 Object Pool

Despite blockwise processing, a certain amount of memory has to be allocated to store intermediary results, before e.g. accumulating them. There are different approaches to handling this. The most trivial implementation would be to always allocate what's required and let Julia's garbage collector free memory once it is no longer needed. While this, given perfect garbage collection, shouldn't change peak memory use of the algorithm, it still brings an overhead of frequently allocating and freeing huge chunks of memory.

A way to optimize this is to preallocate arrays and reuse them across iterations. As most arrays are only required for a short time, it also makes sense to reuse them for different computations, as shown for the `scratch` array in Listing 2.10. However, a single `scratch` array is not sufficient to implement the AMICA algorithm, as several arrays are written and released in an interleaving manner. Manually implementing this using a set of preallocated arrays was tedious and error-prone.

```jl
# allocates "intermed" in each iteration
for i in 1:100
  q = calc_q()
  sum_q = sum(q, dims=1)

  fp = calc_fp()
  sum_fp = sum(fp, dims=1)
end

# preallocate and reuse
q = Array(undef, n, m)
fp = Array(undef, n, m)
for i in 1:100
  q .= calc_q()
  sum_q = sum(q, dims=1)

  fp .= calc_fp()
  sum_fp = sum(fp, dims=1)
end

# reuse scratch array
```

```
scratch = Array(undef, n, m)
for i in 1:100
  scratch .= calc_q()
  sum_q = sum(scratch, dims=1)

  scratch .= calc_fp()
  sum_fp = sum(scratch, dims=1)
end
```

Listing 2.10 – Different ways of managing temporary arrays

To simplify the management of temporary arrays while still avoiding excessive allocations, we implemented a concept called Object Pool. It consists of a struct holding a configurable number of temporary arrays, together with two methods to acquire and release them. When acquired, the array is reshaped to the requested proportion and then blocked for other acquirers. Once released, it is available to be acquired again. The user still needs to make sure that a released array is no longer used after releasing it. To debug such situations, we added a flag which "poisons" all released arrays by writing NaN to them, making it easier to spot operations which operate on a previously released array. While manually managing a scratch array would be simpler in the example shown in Listing 2.11, AMICA.jl requires seven such arrays, and manually managing and reshaping them would require significant effort.

```jl
pool = ObjectPool{T}(n * n, 1) # pool with capacity 1
for i in 1:100
  q = pool_acquire!(pool, (n, n))
  q .= calc_q()
  sum_q = sum(q, dims=1)
  pool_release!(pool, q)

  fp = pool_acquire!(pool, (n, n))
  fp .= calc_fp()
  sum_fp = sum(fp, dims=1)
  pool_release!(pool, fp)
end
```

Listing 2.11 – Using the object pool to manage temporary arrays

2.2.5 Multithreading

Without further adjustments, computations in Julia are not parallelized on the CPU but run within a single thread. In addition, parallelization needs to be structured differently on a CPU, compared to a GPU, as the number of concurrent tasks is significantly lower while the speed of execution is higher. Simply porting all GPU threads to the CPU would most likely lead to suboptimal results, as the per-thread overhead is higher on a CPU, and also the CPU wouldn't be able to apply optimizations like SIMD to the computations. It is therefore desirable to break the algorithm into larger chunks, e.g. equal to the number of processor cores, and run them in parallel.

Blockwise processing already splits the AMICA algorithm into work units of configurable size. It is therefore logical to leverage the block logic for CPU parallelization, by launching a number of threads where each thread independently processes a set of blocks and accumulates their results. As a second step, the per thread results are then accumulated to the final value.

To implement this, we add a thread count variable to the `SingleAmicaStruct` and initialize one Object Pool per thread. Additionally, we need to add a second layer of accumulation. To store the values to be accumulated, we define a new `BlockAccumulators` struct which is filled by each thread.

Finally, we spawn the threads and add accumulation of the resulting `BlockAccumulators`. A schematic example is shown in Listing 2.12, with the variable `dmu_numer_acc` representing one field within the `BlockAccumulators` struct.

```jl
# accumulator with extra per thread dimension
dmu_numer_acc = Array(undef, nthreads, n, m)

# spawn threads
Threads.@threads for tid in 1:nthreads
  for blocks in 1:num_blocks
    range = ((block-1)*block_size+1):(block*block_size)
    scratch .= (...) # some computation
    # accumulate to local accumulator
    dmu_numer_acc[tid, :, :] .+= sum(scratch, dims=1)[1, :, :]
  end
end

# accumulate results
dmu_numer .= sum(dmu_numer_acc, dims=1)[1, :, :]
```

Listing 2.12 – Schematic representation of multithreaded blockwise processing

The introduction of multithreading has negligible impact on the single threaded benchmarks. However, parallelizing AMICA by 64 threads significantly cuts iteration times, roughly by a factor of 8 compared to the single threaded execution. As each thread requires its own object pool, memory consumption rises when the thread count is increased, a phenomenon that is similarly present in AMICA Fortran.

	Memory	Time
CPU	7.5 GB	6.0 s/iter
CPU	3.5 GB	7.1 s/iter
GPU	15.6 GB	0.1 s/iter
CPU	3.6 GB	6.6 s/iter
GPU	0.9 GB	0.5 s/iter
CPU	1.1 GB	4.9 s/iter
CPU*64	2.8 GB	0.6 s/iter

As the object pool represents only a part of the memory use of AMICA.jl, we still see a static part of memory usage and memory therefore does not increase linearly with thread count, but instead thread count can be seen as a multiplier to the block size, accompanied by some overhead due to the thread spawning and the `BlockAccumulators` struct.

2.3 Benchmarks

To assess the effectiveness of our changes, we performed a series of AMICA executions with 259 distinct parameterizations, measuring iteration time after the first iteration, peak CPU memory and peak GPU memory. We ran all benchmarks six times with 40 iterations each, resulting in 1,554 total runs. Unless noted otherwise, we followed the benchmarking methodology described in Chapter 2.1.1. The exact algorithm used to determine the set of parameters is shown in Chapter A.1.

For all of the following benchmarks, we skipped configurations in which not every thread had at least one block (i.e., block size $\leq$ samples / threads), as otherwise some threads would be idle and the degree of parallelization would no longer be comparable. Additionally, we did not benchmark configurations with both multithreading and GPU acceleration enabled, as the GPU already parallelizes operations and there is no added benefit of starting those GPU operations in multiple threads.

Given these benchmark results, we first show that our final implementation is algorithmically correct. We then compare the memory use and runtime of AMICA.jl and AMICA Fortran across several datasets and configurations. Finally, we present an analysis of the performance bottlenecks in our AMICA.jl implementation.

2.3.1 Correctness

To verify correctness of our implementation, we execute AMICA.jl for all three datasets and compare the resulting log-likelihood to the output of AMICA Fortran. The graphs in Figure 2.3 show that the likelihood closely tracks Fortran using both the CPU and GPU backend, with 32-bit and 64-bit precision. We conclude that AMICA.jl now reliably computes results that match the output of the Fortran implementation.

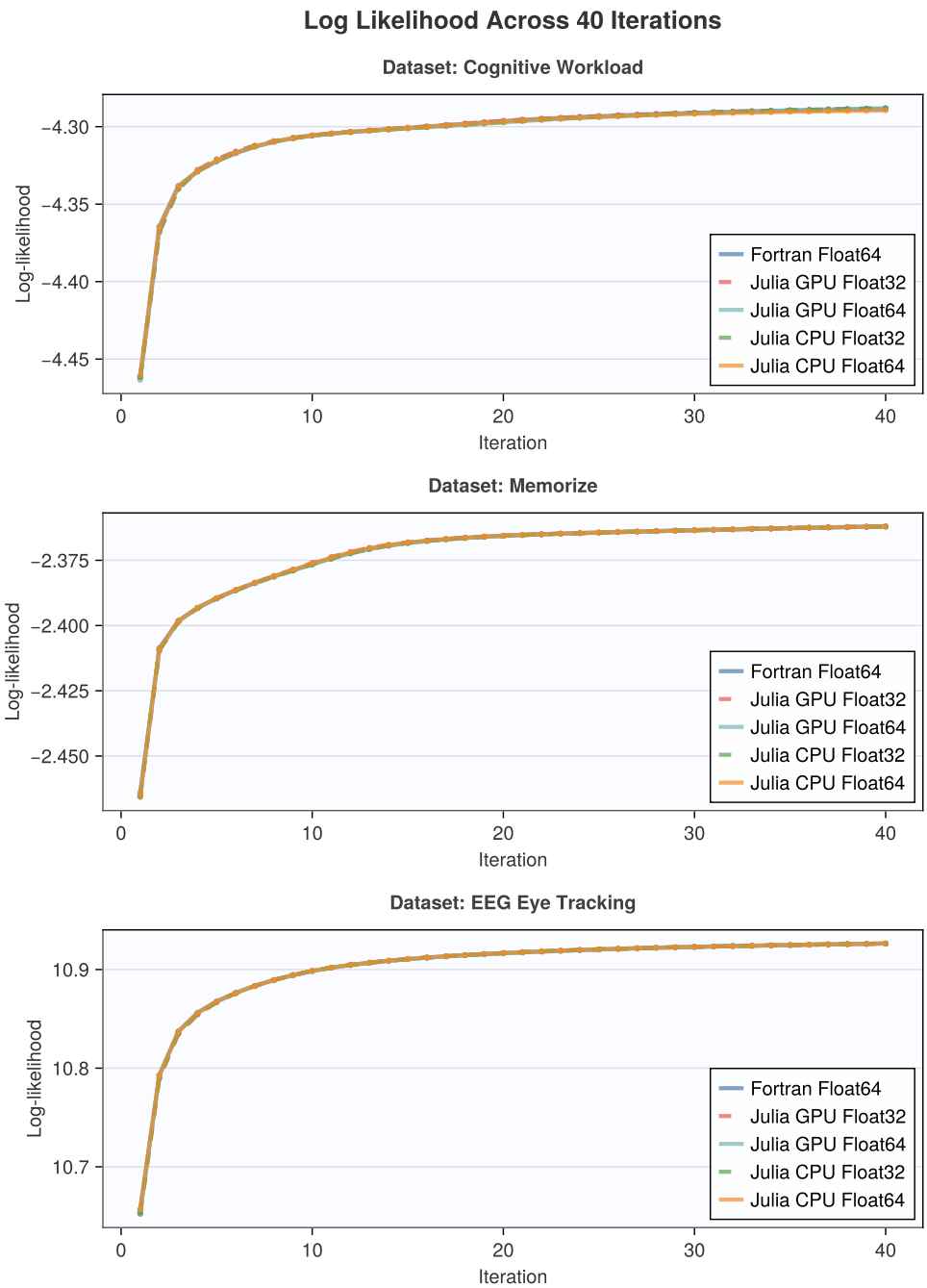

Figure 2.3 – Per iteration log-likelihood by implementation: all implementations closely track the LL of AMICA Fortran

2.3.2 Runtime

The execution time is a key factor in the practical usability of the AMICA algorithm. To achieve good unmixing, 1000 or more iterations often need to be performed, meaning the algorithm can take several hours to complete. Shortening runtime is therefore beneficial and directly reduces the time and, potentially, costs involved in using AMICA.

We present various influencing parameters and a comparison of the different implementations below. The full results of our runtime benchmarks can be found in Chapter A.2.

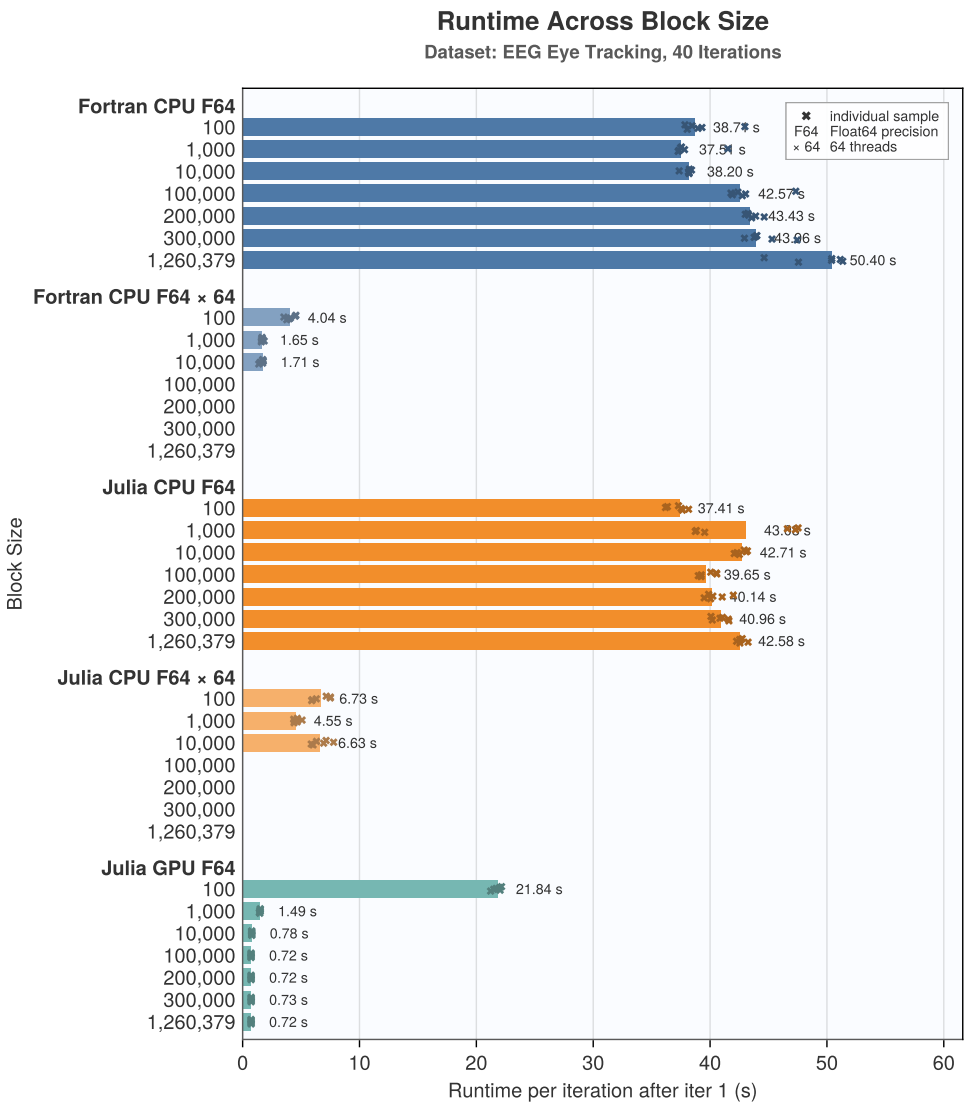

Figure 2.4 – Iteration time by block size. We omit configurations with a *block count* smaller than thread count.

In theory, block size (i.e. number of samples per processed block) should only play a minor role in execution time, as the same operations are performed in a different order. This was largely confirmed by our benchmarks as shown in Figure 2.4. Although there were minor differences in iteration times, we do not consider block size to be the decisive parameter for execution time.

Block size starts to significantly influence runtime once parallelization is enabled. If the block size is smaller than the maximum number of GPU threads, the GPU is not fully utilized and execution is therefore significantly slower, as shown by the benchmarks with block size 100 and 1000.

The same applies with multithreading on the CPU and $\text{block_size} > \frac{\text{n_samples}}{\text{n_threads}}$, which leads to a situation where there are fewer blocks than threads, effectively reducing the parallelism to the number of blocks. We exclude these configurations from the benchmarks because memory usage often prevents execution (since memory depends on $\text{thread_count} * \text{block_size}$). Furthermore, it is clear that an execution with a number of threads equal to the number of blocks will always perform as well or better.

Due to the minor role of block size, we will not list it as a separate dimension in the following benchmarks. Instead, we will select the optimal block size for each parameterization and display only its result.

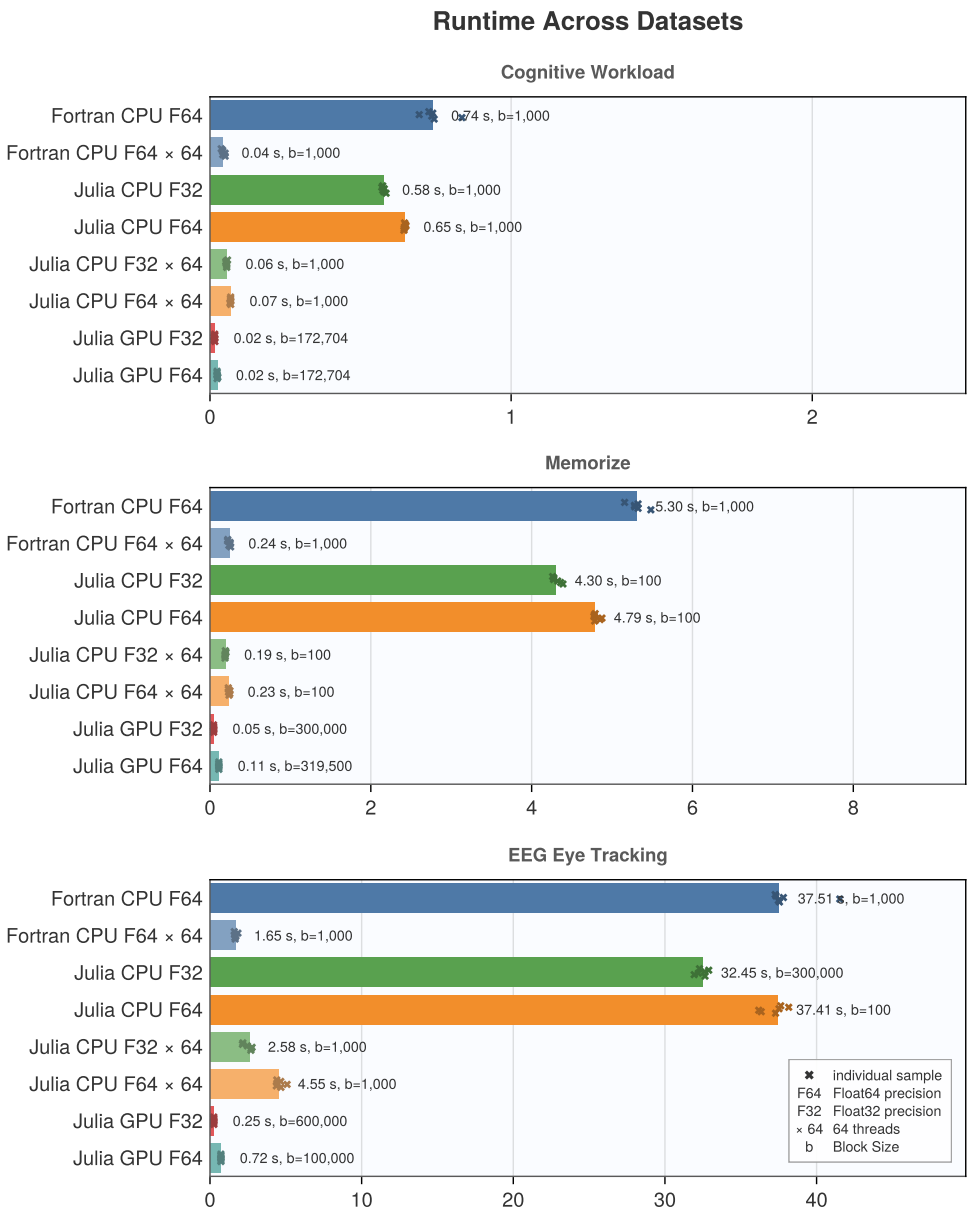

Figure 2.5 – Iteration time across datasets: Runtime increases significantly with problem size, but relative differences between parameterizations remain comparable

In the following, we compare how the implementations react to different problem sizes and how runtime differs between implementations when using the optimal blocksize. Figure 2.5 shows a comparison of AMICA Fortran with

one or 64 threads versus AMICA.jl with one thread, with 64 threads, or with GPU acceleration, each with Float32 data or Float64 data. The comparison reveals a similar picture regardless of dataset size: the fastest implementation is AMICA.jl on the GPU, with a speedup of between 2x and 6.6x compared to multithreaded Fortran. This is followed by the multi-threaded CPU implementations, where the results are mixed. In some cases, AMICA Fortran is faster, and in others, AMICA.jl is faster. However, switching to Float32 in AMICA.jl results in a speed advantage in all cases. Overall, we conclude that AMICA.jl with GPU acceleration outperforms the multi-threaded Fortran implementation in all cases. Furthermore, our implementation of multithreading is effective and delivers similar results to the multi-threaded Fortran code. The runtime of all implementations increases comparably with the size of the dataset.

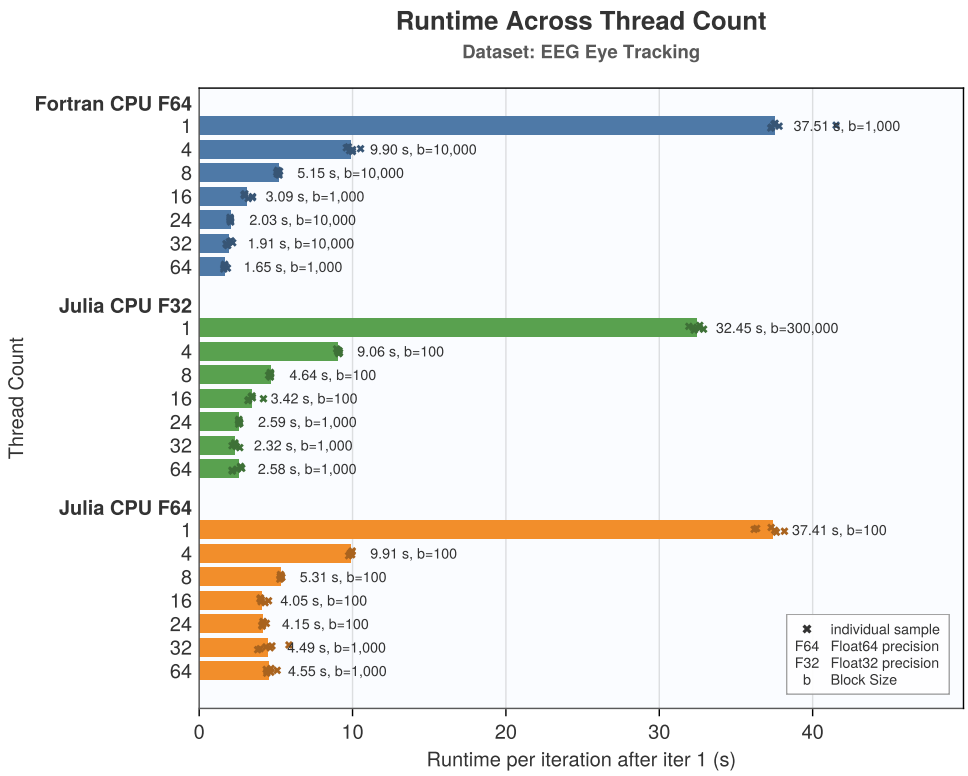

Figure 2.6 – Iteration time by thread count

To examine the effects of multithreading in more detail, we compare the results of AMICA Fortran and AMICA.jl with different numbers of threads in Figure 2.6. The results show that the greatest performance improvement is achieved

between 16 and 24 threads and that further increases have diminishing returns. However, Fortran achieves a greater improvement than Julia when increasing from 16 to 24 threads. Multithreading therefore effectively reduces the runtime of the AMICA algorithm in all implementations, but the effects do not scale indefinitely.

From these results, we conclude that the CPU performance of AMICA.jl is similar to that of AMICA Fortran, multithreading effectively reduces the runtime of AMICA, and GPU acceleration is effective and outperforms all other implementations. The change from 64-bit to 32-bit precision in AMICA.jl is also an effective method of reducing execution time in all configurations. In addition, we have shown that the choice of block size has only a minor effect on the iteration time as long as the block size is small enough to utilize all CPU threads or large enough to utilize all GPU threads.

2.3.3 Memory Usage

The EEG datasets on which AMICA is typically applied can be many megabytes to gigabytes large. Since the algorithm must allocate the entire dataset and additional intermediary results, memory use can become a limiting factor. We thus compared the peak CPU and GPU memory use of the AMICA implementations along three dimensions: block size, problem size and implementation. We omit multithreaded runs from the following benchmarks as the number of threads basically acts as a multiplier to the block size (with some additional overhead). The full memory usage results of our benchmark series can be found in Chapter A.3.

In order to draw a comparison between the different implementations and to demonstrate the effect of individual configuration parameters on memory consumption, we will now present several illustrative comparisons of these results.

To establish a baseline, we first benchmarked memory use with **blockwise processing disabled**. Looking at the results in Figure 2.7, we see that all implementations outperform the 7.5GB of memory AMICA.jl used on the Memorize dataset before this project.

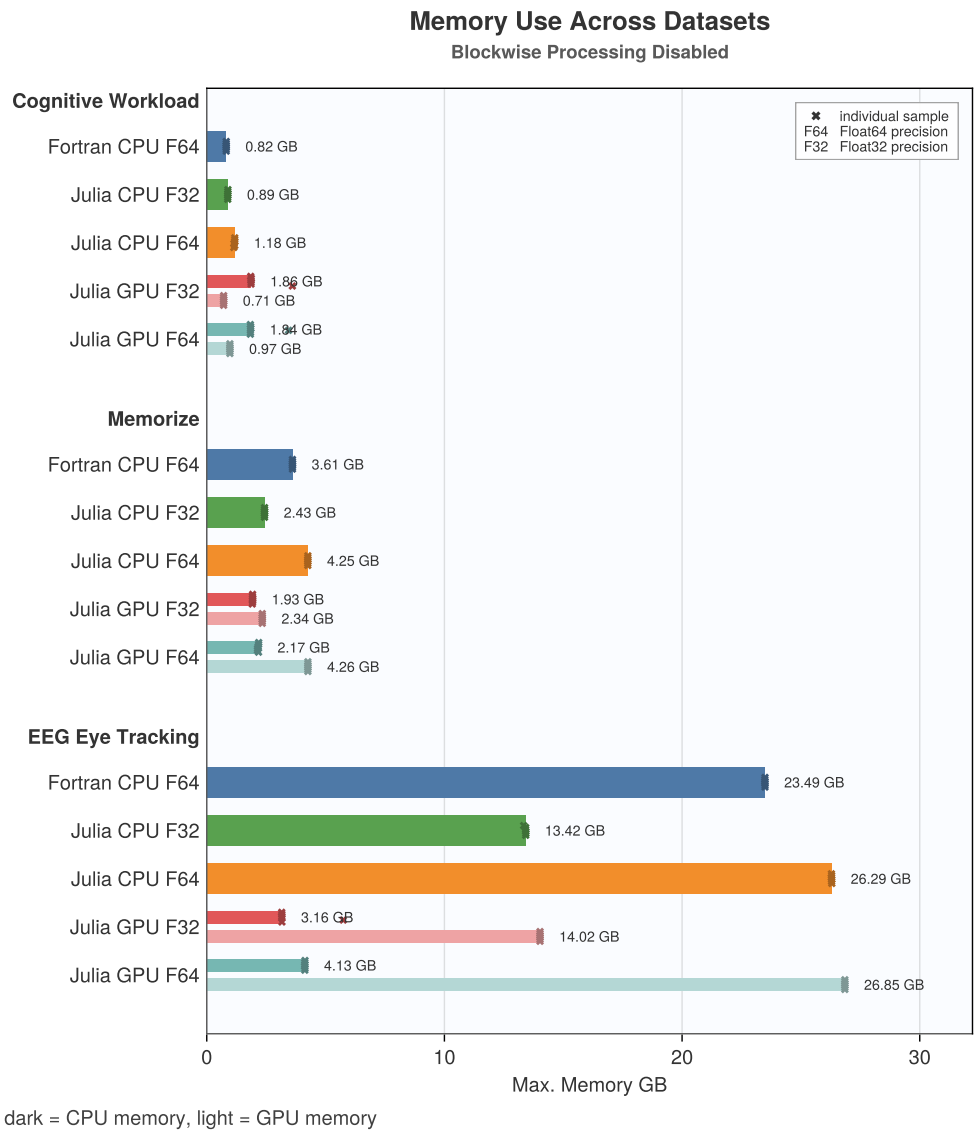

Figure 2.7 – Memory use across datasets without blockwise processing

Fortran is still among the most efficient implementations, only beaten by the Float32 Julia implementation for the two larger datasets. Comparing the Float64 CPU implementations, Fortran is noticeably more memory efficient than Julia, with the offset increasing for larger datasets, and the Julia GPU implementation requiring slightly more memory than the CPU version. When looking at combined CPU and GPU memory use, AMICA.jl requires more memory when running on the GPU than on the CPU. We suspect that the reason is the memory required to load the CUDA library, which already uses over 1GB of memory, as

well as some overhead of loading the dataset to the CPU and then duplicating it to the GPU.

We implemented **blockwise processing** to reduce the memory consumption of our AMICA implementation. In Figure 2.8, we compare the memory used when blockwise processing is enabled, for all three datasets and block size 1000. The dim bars show the values without blockwise processing for comparison. Blockwise processing significantly reduced memory consumption, with the strongest reduction seen on the largest dataset. However, memory usage continues to be influenced by the size of the dataset, even though blocks of identical size are now used regardless of the problem size. We assume that this is due to the initial loading of the data and some computation steps that still operate on the full dataset.

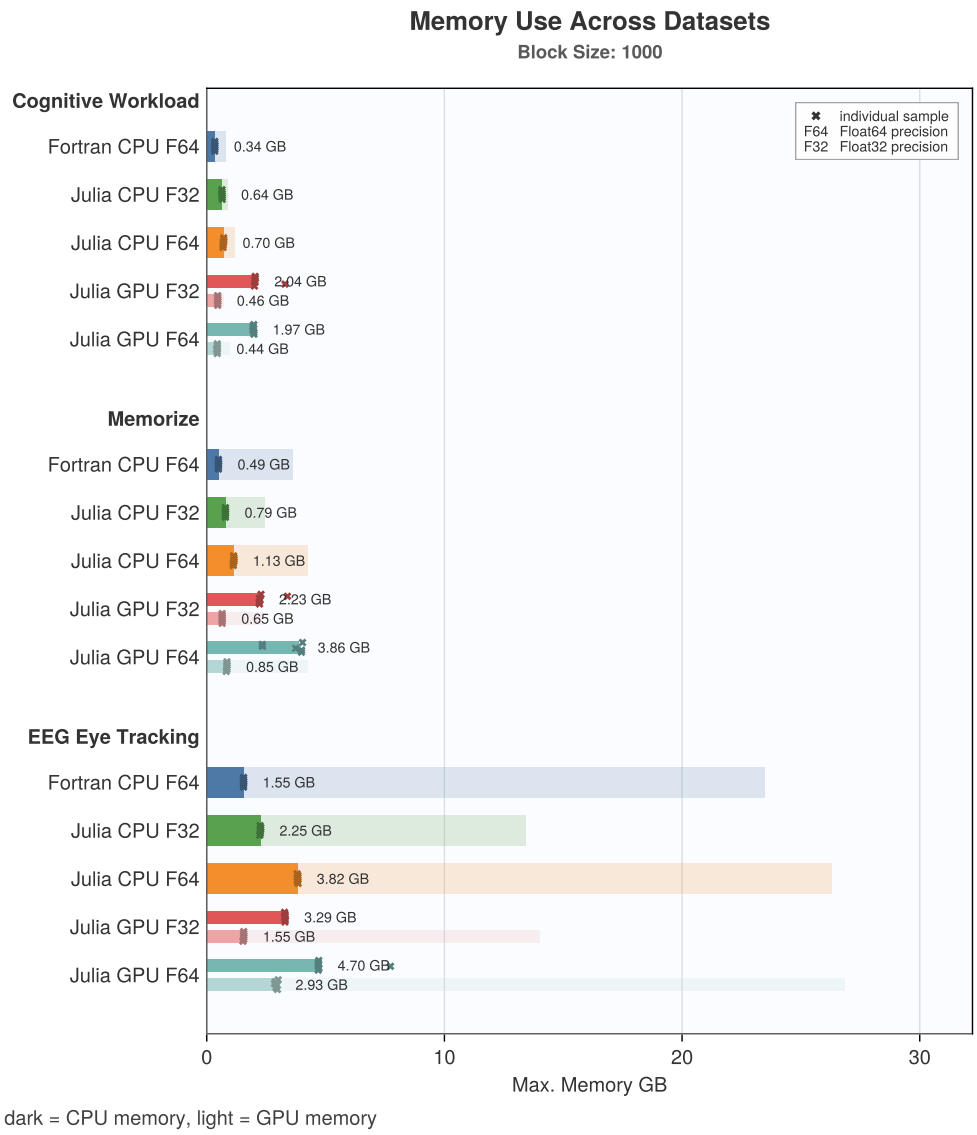

Figure 2.8 – Memory use across datasets with block size 1000, compared to blockwise processing disabled (dim bars)

Setting the correct block size is an effective way to influence AMICA's memory usage. We have therefore evaluated the influence of different block sizes in more detail and show a comparison of different block sizes and the resulting memory usage in Figure 2.9. It shows that the greatest savings are achieved when reducing the block size to 10,000, and that further reductions to 1,000 or 100 actually worsen the results for the Julia implementation. One possible reason for this is that the block loop runs extremely frequently with very small

blocks, meaning that the garbage collector does not clean up often enough to free up memory after each iteration. Memory usage was largely consistent across samples, with the exception of CPU memory for GPU runs, where we saw occasional outliers. We suspect that the reason for those outliers is Julia's non-deterministic garbage collection, but cannot explain why they primarily occur in GPU runs.

Another finding of this analysis is that the CPU memory required for execution on the GPU does not increase with the size of the blocks, but remains constant. The benchmarks also demonstrate that switching to Float32, with or without blockwise processing, leads to a significant reduction in the amount of required memory.

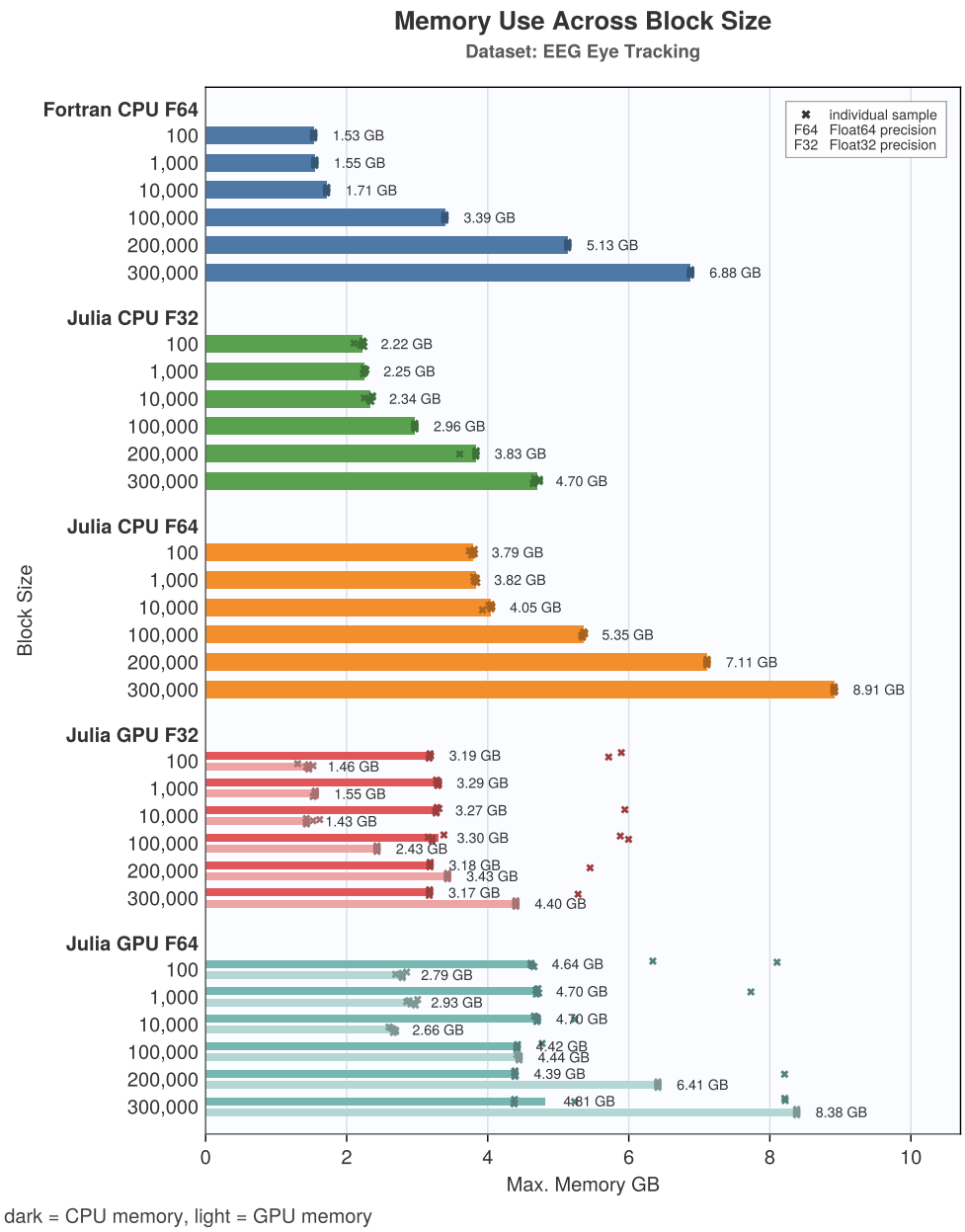

Figure 2.9 – Memory use across block sizes

From these results, we conclude that the changes made to AMICA.jl, in particular the implementation of blockwise processing, have achieved a significant reduction in memory requirements. The memory requirements of AMICA.jl are slightly higher when using Float64 data, but lower than AMICA Fortran when using Float32 data. The required GPU memory is slightly lower than the memory

required by the CPU implementations, but an additional static amount of CPU memory is required to load the dataset and the CUDA library.

		Time			Allocations		
Tot / % measured:		55.1s / 59.8%			3.73GiB / 7.7%		
Section	ncalls	time	%tot	avg	alloc	%tot	avg
update_parameters	5	31.7s	96.2%	6.35s	70.9MiB	24.0%	14.2MiB
process_blocks	5	31.7s	96.2%	6.34s	57.1MiB	19.3%	11.4MiB
y_rho	800	7.57s	23.0%	9.47ms	1.45MiB	0.5%	1.86KiB
source_signals	800	6.09s	18.5%	7.62ms	0.00B	0.0%	0.00B
drho_numer	800	4.32s	13.1%	5.40ms	1.42MiB	0.5%	1.82KiB
log	800	3.93s	11.9%	4.91ms	0.00B	0.0%	0.00B
abs	800	329ms	1.0%	411µs	0.00B	0.0%	0.00B
sum	800	56.2ms	0.2%	70.2µs	1.42MiB	0.5%	1.81KiB
expQ	800	3.90s	11.8%	4.87ms	0.00B	0.0%	0.00B
z	800	2.73s	8.3%	3.42ms	0.00B	0.0%	0.00B
dmu_denom	800	1.38s	4.2%	1.72ms	1.45MiB	0.5%	1.86KiB
dA	800	1.02s	3.1%	1.27ms	30.9MiB	10.5%	39.6KiB
calc	800	1.01s	3.1%	1.27ms	0.00B	0.0%	0.00B
z_norm	800	630ms	1.9%	788µs	0.00B	0.0%	0.00B
dmu_numer	800	544ms	1.7%	680µs	1.42MiB	0.5%	1.81KiB
fp	800	410ms	1.2%	513µs	37.5KiB	0.0%	48.0B
dbeta_denom	800	292ms	0.9%	365µs	1.45MiB	0.5%	1.86KiB
newton_sigma2	800	289ms	0.9%	361µs	12.0MiB	4.1%	15.3KiB
dlambda_numer	800	264ms	0.8%	331µs	1.42MiB	0.5%	1.81KiB
Q	800	250ms	0.8%	312µs	37.5KiB	0.0%	48.0B
sum	800	233ms	0.7%	291µs	0.00B	0.0%	0.00B
Qmax	800	224ms	0.7%	281µs	0.00B	0.0%	0.00B
kp	800	187ms	0.6%	233µs	1.42MiB	0.5%	1.81KiB
y	800	143ms	0.4%	178µs	0.00B	0.0%	0.00B
sum_z	800	79.4ms	0.2%	99.2µs	1.42MiB	0.5%	1.81KiB
Lt	800	24.2ms	0.1%	30.2µs	0.00B	0.0%	0.00B
qconst	800	8.37ms	0.0%	10.5µs	1.42MiB	0.5%	1.81KiB
accumulate	5	15.3ms	0.0%	3.05ms	12.4MiB	4.2%	2.49MiB
ldet	5	2.88ms	0.0%	576µs	398KiB	0.1%	79.7KiB
myAmica.dA	5	505µs	0.0%	101µs	594KiB	0.2%	119KiB
myAmica.LL	5	267µs	0.0%	53.4µs	144B	0.0%	28.8B
para	5	76.5µs	0.0%	15.3µs	33.8KiB	0.0%	6.75KiB
shape	5	64.1µs	0.0%	12.8µs	0.00B	0.0%	0.00B
loc	5	14.1µs	0.0%	2.82µs	0.00B	0.0%	0.00B
prop	5	5.62µs	0.0%	1.12µs	0.00B	0.0%	0.00B
scale	5	5.57µs	0.0%	1.11µs	0.00B	0.0%	0.00B
sphering	1	625ms	1.9%	625ms	174MiB	58.7%	174MiB
removeMean	1	358ms	1.1%	358ms	12.1MiB	4.1%	12.1MiB
update_mixing	5	158ms	0.5%	31.7ms	8.59MiB	2.9%	1.72MiB
do_newton!	5	158ms	0.5%	31.7ms	8.59MiB	2.9%	1.72MiB
kernel	5	158ms	0.5%	31.5ms	8.01MiB	2.7%	1.60MiB
init pools	1	66.2ms	0.2%	66.2ms	29.3MiB	9.9%	29.3MiB
init proportions	1	18.6ms	0.1%	18.6ms	507KiB	0.2%	507KiB
init A	1	17.4ms	0.1%	17.4ms	448KiB	0.1%	448KiB
reparameterize	5	253µs	0.0%	50.5µs	205KiB	0.1%	41.0KiB
init location	1	14.1µs	0.0%	14.1µs	3.62KiB	0.0%	3.62KiB
init scale	1	7.82µs	0.0%	7.82µs	5.44KiB	0.0%	5.44KiB
initialize_shape_parameter!	1	7.65µs	0.0%	7.65µs	0.00B	0.0%	0.00B
calculate_DLL	5	6.25µs	0.0%	1.25µs	0.00B	0.0%	0.00B
movetogpu	1	40.0ns	0.0%	40.0ns	0.00B	0.0%	0.00B

Figure 2.10 – Performance breakdown of a single threaded execution of AMICA.jl

2.3.4 Bottlenecks

In addition to the aforementioned performance benchmarks, we will now discuss the output of the fine-grained benchmarks obtained by integrating the `TimerOutputs.jl` library. Figure 2.10 shows the evaluation of four single-threaded iterations of AMICA.jl with block size 5000.

The output shows that most of the runtime is spent in the `update_parameters` function and, within that, in the `process_blocks` function. This is relevant because it is the part parallelized by our implementation of multithreading. When analyzing the top four areas (`y_rho`, `source_signals`, `drho_numer`, `expQ`), we see that the dominant operations perform `log`, `exp` and `mul` operations on arrays with large dimensions. We therefore conclude that we were able to eliminate most other overhead and that these operations are now the main limiting factor in further improving the implementation's runtime.

We additionally note that only around 8 percent of the memory is allocated within the measured areas, which means that most allocations happen outside of AMICA's main loop, most likely during construction of the `SingleModelAmica` object. This shows that we successfully reduced repeated allocations and instead pre-allocate the required memory.

```jl
# y_rho
y_rho .= exp.((myAmica.shape .- 1.0) .* log.(abs.(y)))

# source_signals
mul!(source_signals, data, W')

# drho_numer → log
z .* log.(scratch) .* scratch

# expQ
expQ .= exp.(Q .- Qmax)
```

Listing 2.13 – Computations within the slowest operations (simplified)

3 Discussion

We now discuss the practical and theoretical results of our work and address the research question of whether an optimized version of AMICA.jl can match the Fortran reference in terms of correctness and performance.

3.1 Correctness and Convergence

By adding a comprehensive test suite and adjusting the AMICA.jl implementation in around 15 aspects, we aligned the results to AMICA Fortran and then verified that alignment by comparing the LL for three different datasets. We regard the matching LL as a clear proxy for correctness and consider this the most important achievement of our work, as it is the foundation of the practical usability of AMICA.jl and of all performance benchmarks, since only a benchmark of a correct algorithm allows for a meaningful comparison with the reference.

The delayed convergence and problems with numerical stability indicated that there were not only minor bugs in AMICA.jl, but also fundamental differences in the implementation. We see this confirmed in the number and scope of the changes we made, which required us to completely rewrite entire functions. At the same time, this also shows that AMICA's results are sensitive to small

differences in implementation, and that maintaining a correct implementation is therefore important. The test suite can also ensure this for future adjustments.

3.2 Runtime

Our implementation of GPU support and multithreading, as well as the benchmarks discussed above, show that AMICA contains sufficient parallel work to be effectively parallelized across multiple processor cores or even a GPU. We thus demonstrated that implementing GPU support in AMICA is possible and effective, and that this can achieve iteration times that surpass AMICA Fortran.

When comparing the impact of different measures, our benchmarks show that parallelization on the GPU is the strongest contributor to runtime improvement. Multithreading on the CPU also proved effective, but showed diminishing returns beyond 16 threads and could not match the performance of the GPU implementation.

Comparing our implementation with AMICA Fortran, we therefore conclude that AMICA.jl is competitive on the CPU and measurably exceeds the reference implementation's performance when running on the GPU.

By improving numerical stability in some areas, we have ensured that AMICA.jl reliably calculates the same results as AMICA Fortran, even when using Float32 data types. This provides an additional performance advantage without noticeable downsides. However, we cannot rule out with certainty that certain data sets, such as very small problems, might still result in numerical issues. Therefore, it should be determined on a case-by-case basis whether using 32-bit precision is sufficient.

3.3 Memory

In addition to runtime, memory consumption is the other constraining factor for AMICA. Memory is typically limited, especially on the GPU, so economical memory usage provides a significant advantage in the practical usability of AMICA.jl.

By implementing blockwise processing, we were able to achieve a drastic reduction in the memory requirements of AMICA.jl, especially for larger datasets.

Memory usage is now no longer only determined by the size of the dataset, but can be influenced by choosing an appropriate block size. However, the size of the dataset continues to play a role in memory consumption, suggesting that some parts of the implementation still operate on the entire dataset or incur a fixed overhead.

We have shown that block size generally plays a minor role in runtime, but in combination with parallelization on the CPU or GPU, there can be a trade-off between runtime and block size, and thus memory consumption. When running AMICA.jl on the CPU, we found that large block sizes can result in not every thread having a block to compute, thereby reducing the effective parallelism to the number of blocks. When running on the GPU, small block sizes reduce parallelism because there is not enough data being processed simultaneously to utilize all GPU threads. Finally, very small block sizes can always cause a certain amount of overhead, which is a disadvantage compared to larger blocks.

Compared to Julia, AMICA Fortran remains the most memory efficient implementation when working with 64-bit data. Julia became more competitive by our changes, and surpassed Fortran when working with 32-bit precision. The GPU acceleration requires a similar amount of GPU memory compared to CPU only executions, but also allocates a static overhead of CPU memory.

3.4 Practical implications

Our improvements are not only theoretical but make a significant difference in the practical applicability of AMICA.jl, for example in EEG workflows. The correctness of the implementation ensures an unmixing of the data that is comparable to the Fortran reference implementation and thus forms the basis for the practical applicability.

Shorter iteration times lead to a noticeable reduction in evaluation time, and potentially cost, especially since AMICA is typically executed for hundreds or thousands of iterations. For example, 1000 iterations on the “EEG Eye Tracking” dataset require about 4:10 minutes using GPU acceleration and 32-bit precision, compared to 25 minutes required in the fastest AMICA Fortran configuration. Since typical consumer PCs often have significantly fewer than 64 cores, we believe it is likely that the practical difference will be even greater.

In EEG studies, it is common to analyze not just a single EEG recording but several subjects. A common approach is to run several AMICA processes simultaneously, as this results in significantly more efficient parallelization than multithreading within the implementation can offer. In those settings, memory can be a key factor in scalability, and therefore our improvements in memory usage can increase the practically achievable degree of parallelization. For example, when running AMICA.jl (32-bit precision) on 16 subjects, given sufficient memory available, we could run all 16 subjects in parallel with four threads on our 64 core machine. This would result in a theoretical runtime of around 9:25 minutes per AMICA, which significantly beats the 43 minutes required for a single 64 thread AMICA. This example shows that efficient memory use can enable significantly more efficient parallelism on the CPU compared to multithreading within the implementation, but GPU acceleration is still twice as fast as the given example.

Our implementation, however, is attractive not only because of its speed, but also because it is easier to understand, maintain, and extend than the Fortran code. AMICA.jl therefore provides a useful compromise between accessibility and high performance.

3.5 Limitations

3.5.1 Gaps in the Implementation

While we believe this work made significant improvements to AMICA.jl, we also want to recognize the following limitations of our findings.

First, we would like to note that although AMICA.jl implements the basic algorithm, it currently omits some notable features. AMICA Fortran offers multi-model functionality, where different models are active for each sample, which provides the benefit of supporting non-stationary settings. This feature used to be present in AMICA.jl but remains to be re-implemented with appropriate unit-tests. AMICA Fortran also includes functionality to choose the optimal block size, a helpful feature that makes it easier to find the optimal parametrization, and likelihood-based data-rejection which automatically rejects samples of low likelihood. AMICA.jl currently also omits the multi-node processing that Fortran implements using the MPI library, and which allows splitting larger chunks of work across multiple machines, possibly increasing the amount of usable compute.

With regards to performance, while we are able to match AMICA Fortran performance using GPU acceleration or 32-bit precision, AMICA.jl is still slightly behind Fortran in regular single- or multi-threaded executions. We see no systematic reason why a Julia implementation couldn't exactly match the Fortran performance, so future work might explore how those remaining gaps might be closed.

Our bottleneck analysis showed that we were able to mitigate most overheads in AMICA.jl, so the current constraining factors are certain mathematical operations like `log`, `exp` or `mul`. Future work could focus on ways to specifically improve those parts, for example by implementing the Intel Vector Math library, previously called Intel MKL, as supported by the Fortran implementation.

The first iteration of AMICA.jl continues to be considerably slower than the subsequent iterations; we typically saw static offsets of around 10s for GPU runs and around 0.1s for CPU runs, a fact that we omit in our benchmarks. We suspect this might be due to Julia's just-in-time compilation and type inference, and thus it is unclear to which degree this could be improved. In addition, we tested GPU acceleration only on the CUDA and Metal backend and have to acknowledge that support is much more limited on Metal, primarily restricted by the feature set of the Metal.jl library.

While our test suite compares several intermediary variables and outputs, and runs with and without Newton Method, we acknowledge that it currently does not cover all code paths; for example, it does not test a fallback from the Newton Method to the natural gradient.

3.5.2 Gaps in the Benchmark

While we ran a large number of distinct benchmark configurations, and repeated each benchmark six times to determine the median result, we still covered only a limited number of influencing factors and parameters. The selection of data sets was limited to three, we only tested AMICA Fortran in the MKL variant (recommended in private communication with Jason Palmer), and we also excluded multi-node support. In the analysis of our benchmarks we omitted the influence of multithreading on memory use, which mainly acts as a multiplier to the block size, but also brings some additional overhead which could be further explored. Since we ran all the benchmarks on an AMD CPU, we cannot rule out that the Intel MKL library used within the Fortran implementation would perform better on Intel CPUs.

Since all benchmarks were run on the same hardware setup, their results shouldn't be generalized without caution. Although comparisons between different implementations are feasible, comparisons between CPU and GPU are only of limited meaning, as their relationship is highly dependent on the respective hardware: For example, the acceleration provided by the GPU compared to the CPU could vary significantly with a different GPU model. It should also be noted that we used a shared server, so the influence of other users' workloads cannot be ruled out.

4

Summary

Our work evolved AMICA.jl from a promising but unreliable re-implementation into a credible alternative that is competitive in terms of computational performance. We achieved this improvement through a combination of algorithmic correctness, improvements to numerical stability, restructuring to improve memory usage, and two forms of parallelization.

By adding a comprehensive test suite, we ensured that regressions introduced during our work or by future changes are identified quickly. Extensive benchmarks allow performance to be measured in terms of timing and memory allocation on a per-function basis.

We showed that adding GPU acceleration to the AMICA algorithm is feasible and that it significantly improves runtime. We implemented blockwise processing as a measure to reduce memory usage and added multithreaded CPU execution. These new features specifically lower the barrier to running AMICA.jl on large EEG datasets.

By increasing numerical stability, mainly by clamping very small intermediate values to a small epsilon, AMICA.jl can now work with 32-bit precision while still

reliably computing a comparable unmixing, further lowering resource usage and offering a capability not present in AMICA Fortran.

Our work therefore substantially improves the practical usability of AMICA.jl and positions it as a more accessible high-performance alternative to the reference Fortran implementation.

Bibliography

AMDGPU.jl. Retrieved August 14, 2025, from https://github.com/JuliaGPU/AMDGPU.jl°

AMICA - Matlab Implementation. Retrieved August 16, 2025, from https://github.com/sccn/BCILAB/blob/master/dependencies/amica.m°

Artifact removal in EEGLAB using ICA. Retrieved February 26, 2026, from https://eeglab.org/tutorials/06_RejectArtifacts/RunICA.html°

Atomix.jl. Retrieved March 1, 2026, from https://juliaconcurrent.github.io/Atomix.jl°

Barras, M., & Booth, L. (2026). *Cognitive Workload 8-level arithmetic* [Computer software]. DOI: 10.18112/openneuro.ds007262.v1.0.2°

Carlsson, K. *TimerOutputs.jl*. Retrieved February 28, 2026, from https://github.com/KristofferC/TimerOutputs.jl°

Choi, S., Cichocki, A., Park, H.-M., & Lee, S.-Y. (2005). Blind source separation and independent component analysis: A review. *Neural Information Processing-Letters and Reviews*, *6*(1), 1–57.

CUDA.jl. Retrieved August 15, 2025, from https://github.com/JuliaGPU/CUDA.jl°

Delorme, A., Palmer, J., Onton, J., Oostenveld, R., & Makeig, S. (2012). Independent EEG sources are dipolar. *Plos One*, *7*(2), e30135.

Julia for HPC - GPU programming. Retrieved August 15, 2025, from https://enccs.github.io/julia-for-hpc/GPU/°

KernelAbstractions.jl. Retrieved August 15, 2025, from https://juliagpu.github.io/KernelAbstractions.jl/stable/°

Lee, T.-W. (1998). *Independent Component Analysis: Theory and Applications*. Springer.

Lulkin, A. (2023). *Reimplementation of AMICA in Julia*.

Naik, G. R., & Kumar, D. K. (2011). An overview of independent component analysis and its applications. *Informatica*, *35*(1).

Palmer, J. A. *AMICA website*. Retrieved August 13, 2025, from https://sccn.ucsd.edu/~jason/amica_web.html°

Palmer, J. A., Kreutz-Delgado, K., & Makeig, S. (2012). AMICA: An adaptive mixture of independent component analyzers with shared components. *Swartz Center for Computational Neuroscience, University of California San Diego, Tech. Rep*, *35*.

Pearlmutter, B., & Parra, L. (1996). Maximum likelihood blind source separation: A context-sensitive generalization of ICA. *Advances in Neural Information Processing Systems*, *9*, 99–101.

Raimondo, F., Kamienkowski, J. E., Sigman, M., & Fernandez Slezak, D. (2012). CUDAICA: GPU optimization of infomax-ICA EEG analysis. *Computational Intelligence and Neuroscience*, *2012*(1), 206972.

Schepers, J., Marathe, M., Manpa, B., & Ehinger, B. V. (2026). *An EEG-EyeTracking unrestricted viewing dataset on MSCOCO* [Computer software].

Stone, J. V. (2004). *Independent Component Analysis: A Tutorial Introduction*. MIT Press.

Urigüen, J. A., & Garcia-Zapirain, B. (2015). EEG artifact removal—state-of-the-art and guidelines. *Journal of Neural Engineering*, *12*(3), 31001.

Yu, X., Hu, D., & Xu, J. (2013). *Blind Source Separation*. John Wiley & Sons.

A

Appendices

A.1 Parameterizations to be Benchmarked

We ran benchmarks for the combination of the following parameters:

- Implementation: Fortran, Julia
- Backend: CPU, GPU
- Datasets: “Cognitive Workload”, “Memorize”, “EEG Eye Tracking”
- Block sizes: 100, 1000, 10000, 100000, 200000, 300000, 600000, “full dataset”
- Precision: 32-bit, 64-bit
- Thread count: 1, 4, 8, 16, 24, 32, 64
- Number of iterations: 40
- Number of runs: 6

We exclude all parameterizations which match one of the following conditions:

- $\text{Block size} > \text{Problem size}$
- $\text{Block size} * \text{Thread count} > \text{Problem size}$
- $\text{Precision} = \text{float32}$ and $\text{Implementation} = \text{fortran}$
- $\text{Backend} = \text{GPU}$ and $\text{Implementation} = \text{fortran}$

A.2 Full Results of the Runtime Benchmarks

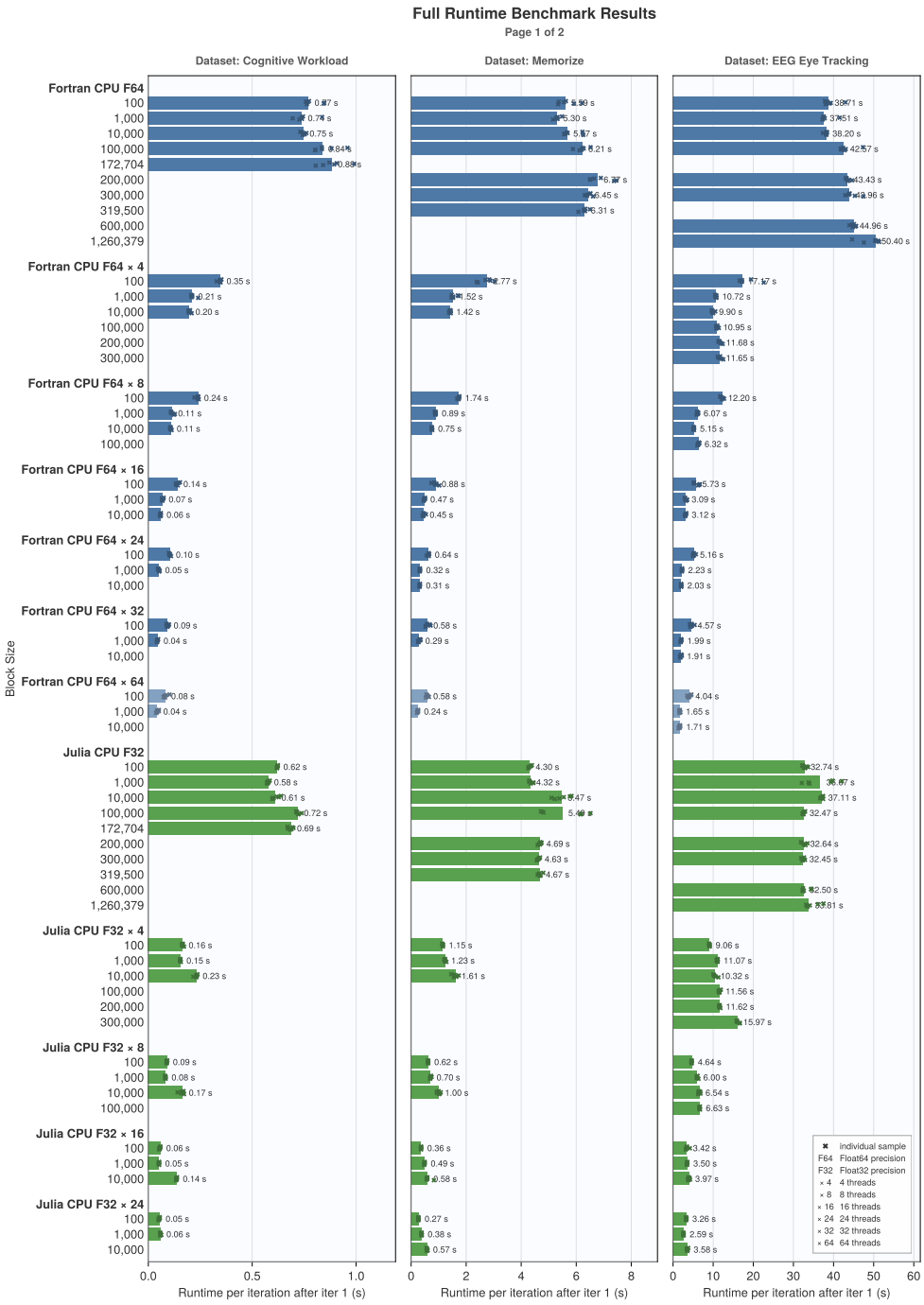

Figure 1.1 – Runtime by AMICA implementation and parametrization (1/2)

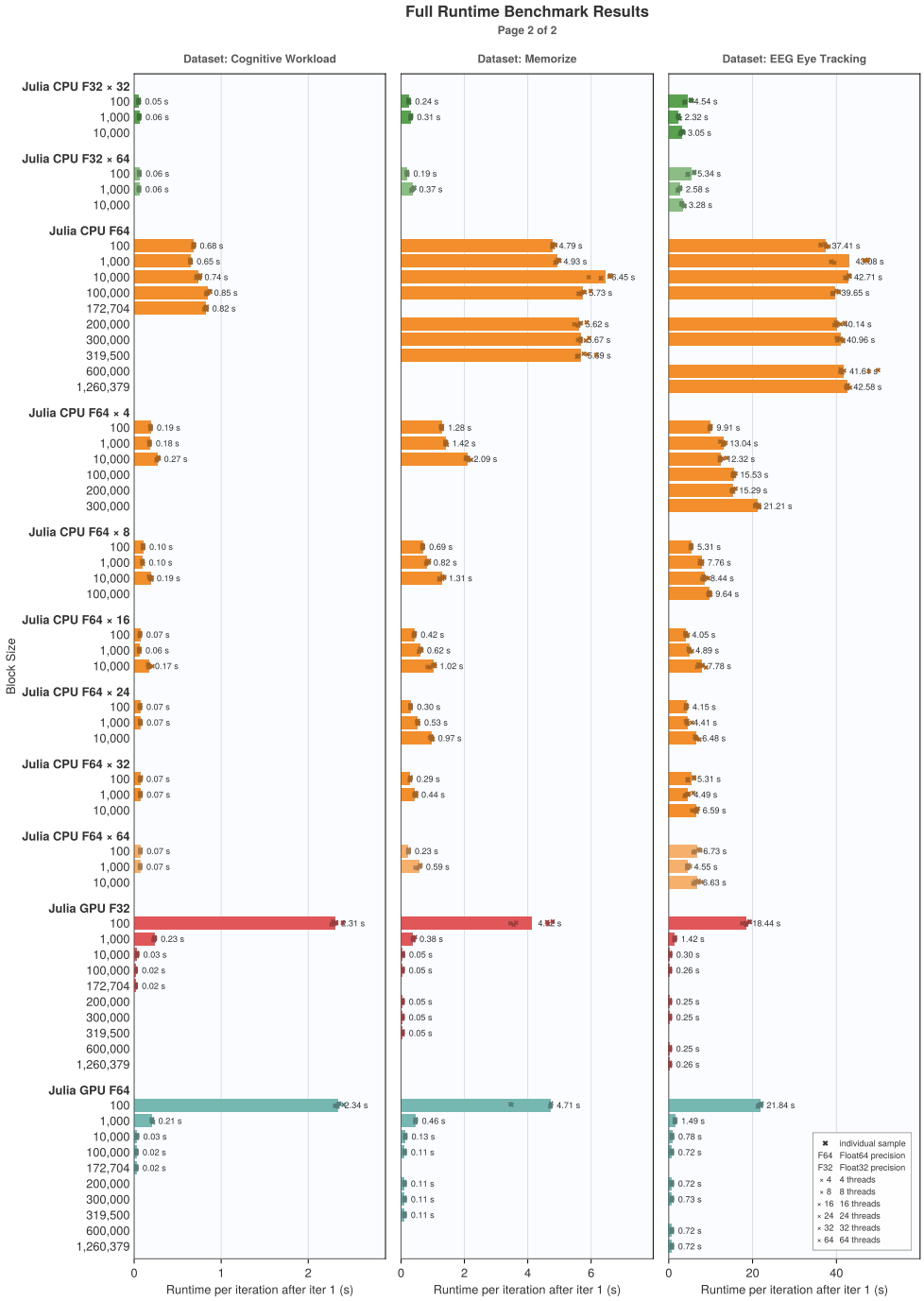

Figure 1.2 – Runtime by AMICA implementation and parametrization (2/2)

A.3 Full Results of the Memory Benchmarks

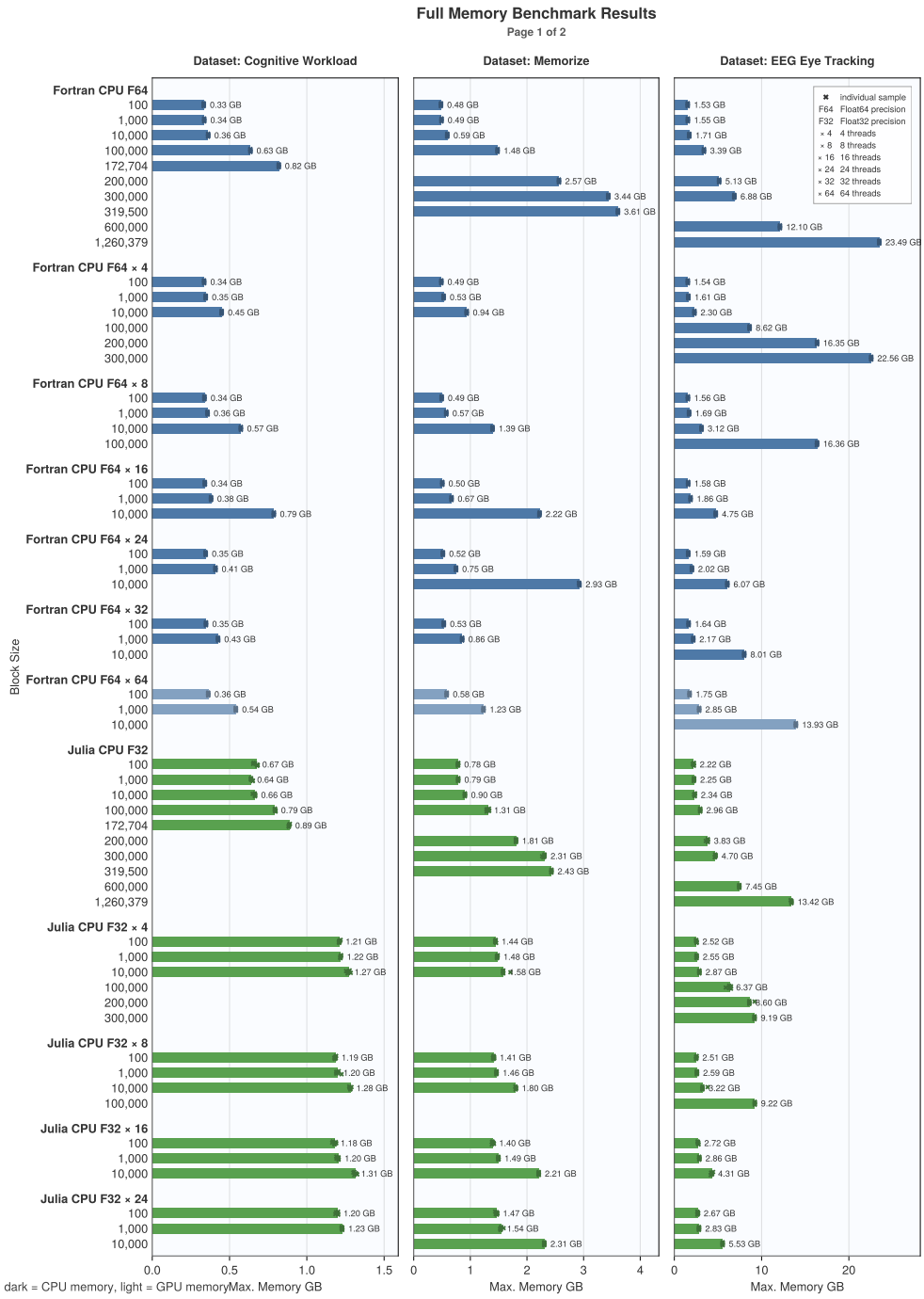

Figure 1.3 – Memory by AMICA implementation and parametrization (1/2)

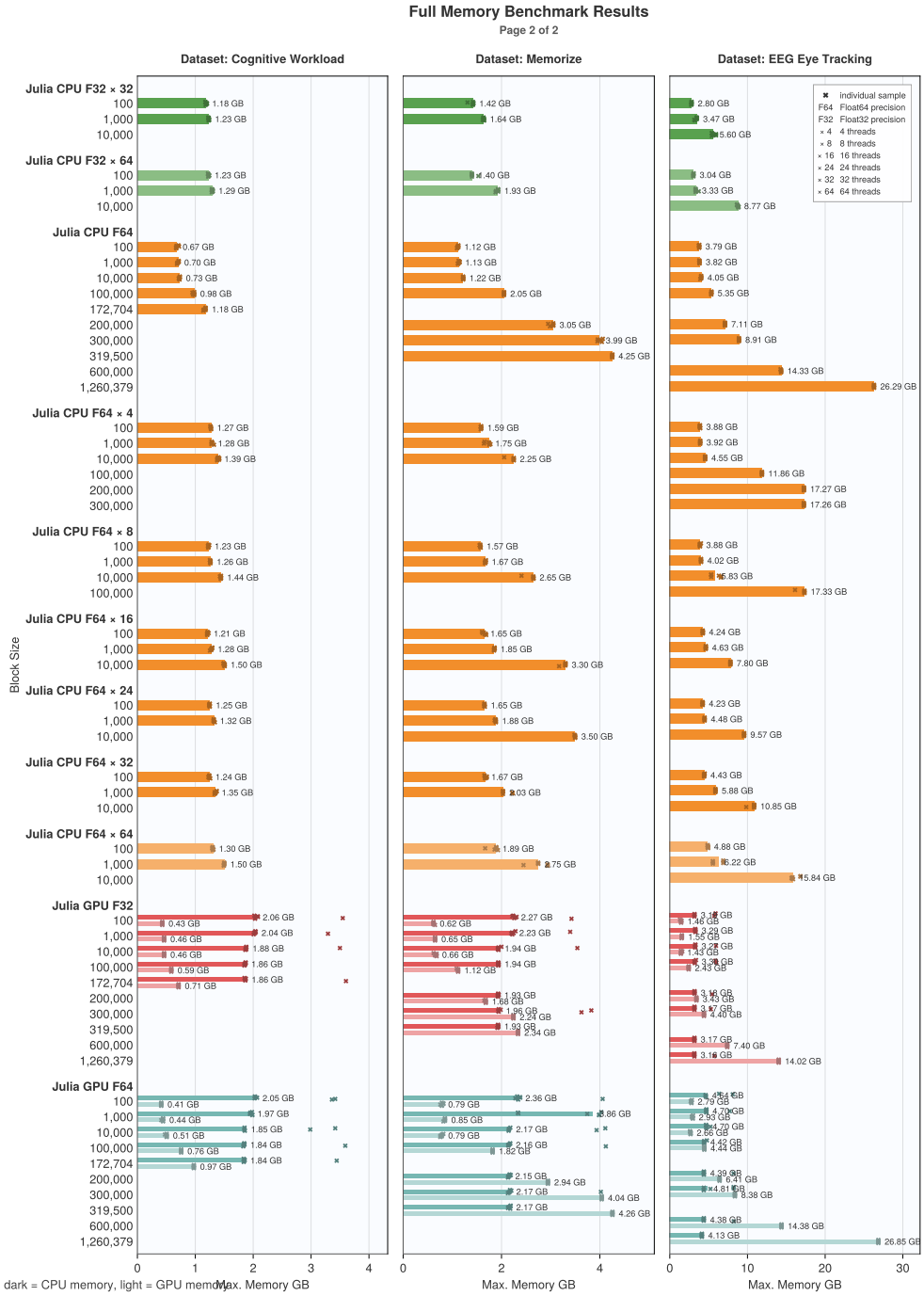

Figure 1.4 – Memory by AMICA implementation and parametrization (2/2)

Declaration

I hereby declare that the work presented in this thesis is entirely my own. I did not use any other sources and references than the listed ones. I have marked all direct or indirect statements from other sources contained therein as quotations. Neither this work nor significant parts of it were part of another examination procedure. I have not published this work in whole or in part before. The electronic copy is consistent with all submitted hard copies.

Date

Signature